

Transformers

third stage of the sequential data

recall:
first is vanilla rnn
second is the lstm and gru

Agenda

Seq-to-Seq Models

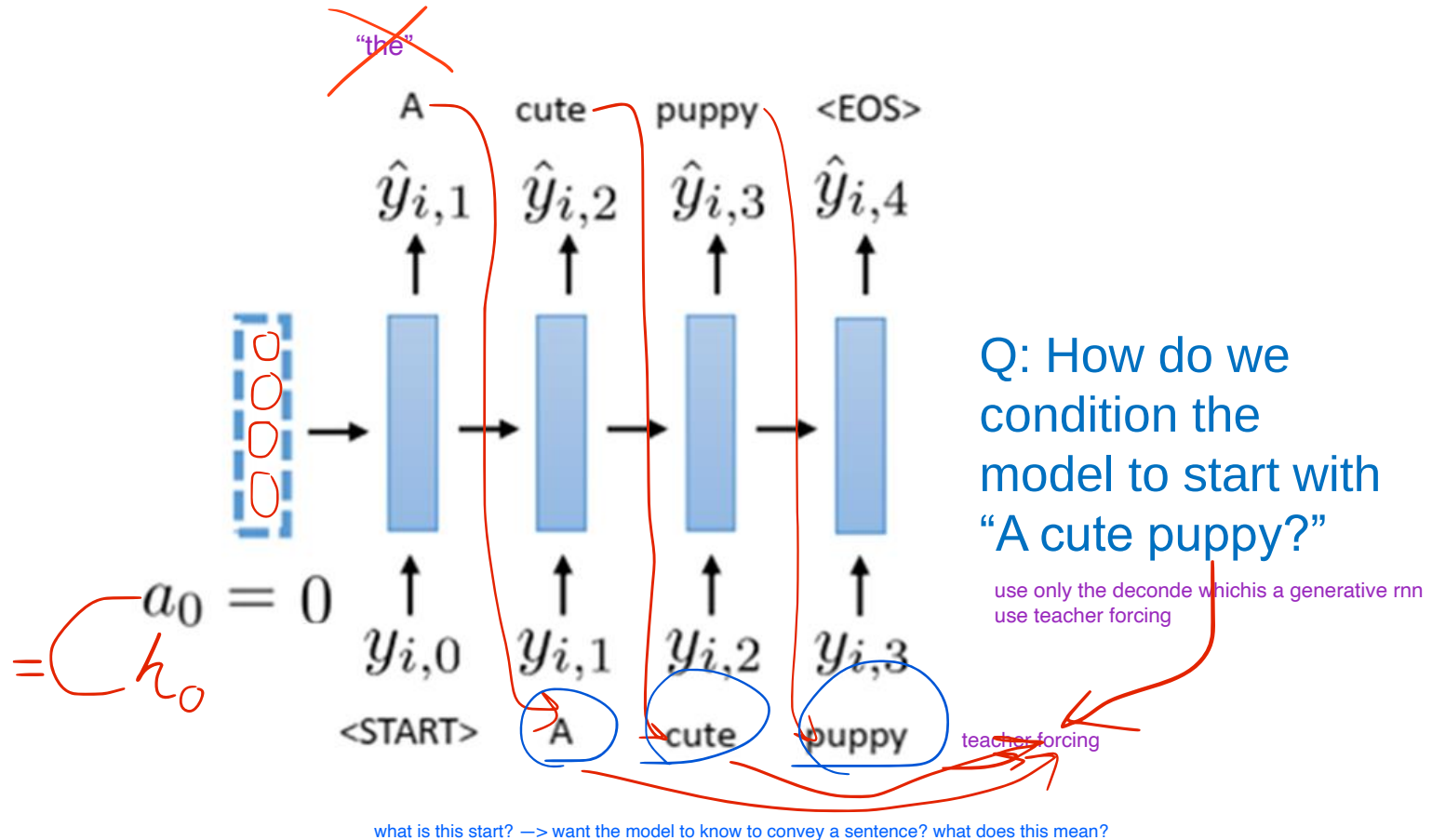
Attention

Transformers

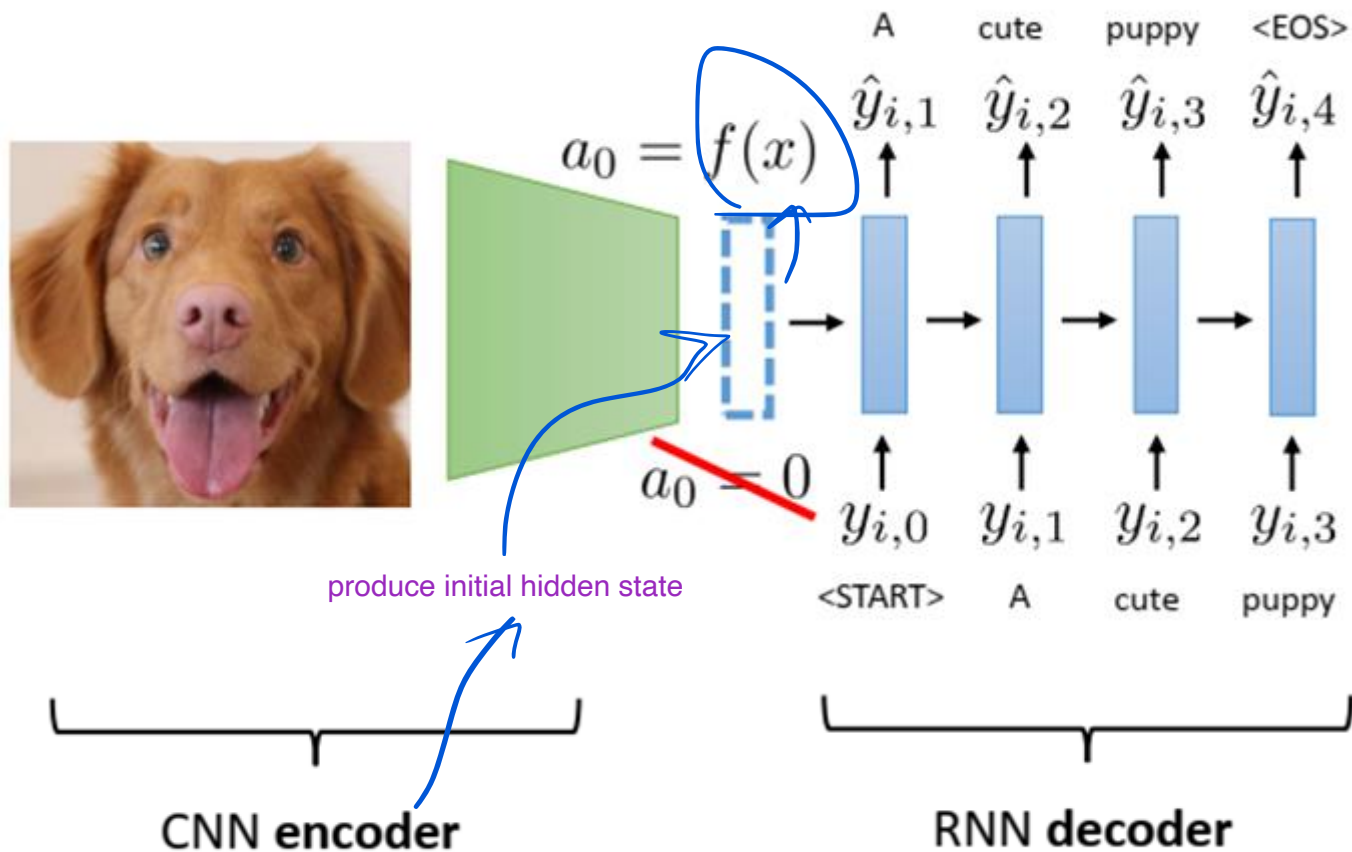
Large Language Models

Examples

Recap: Generative RNN



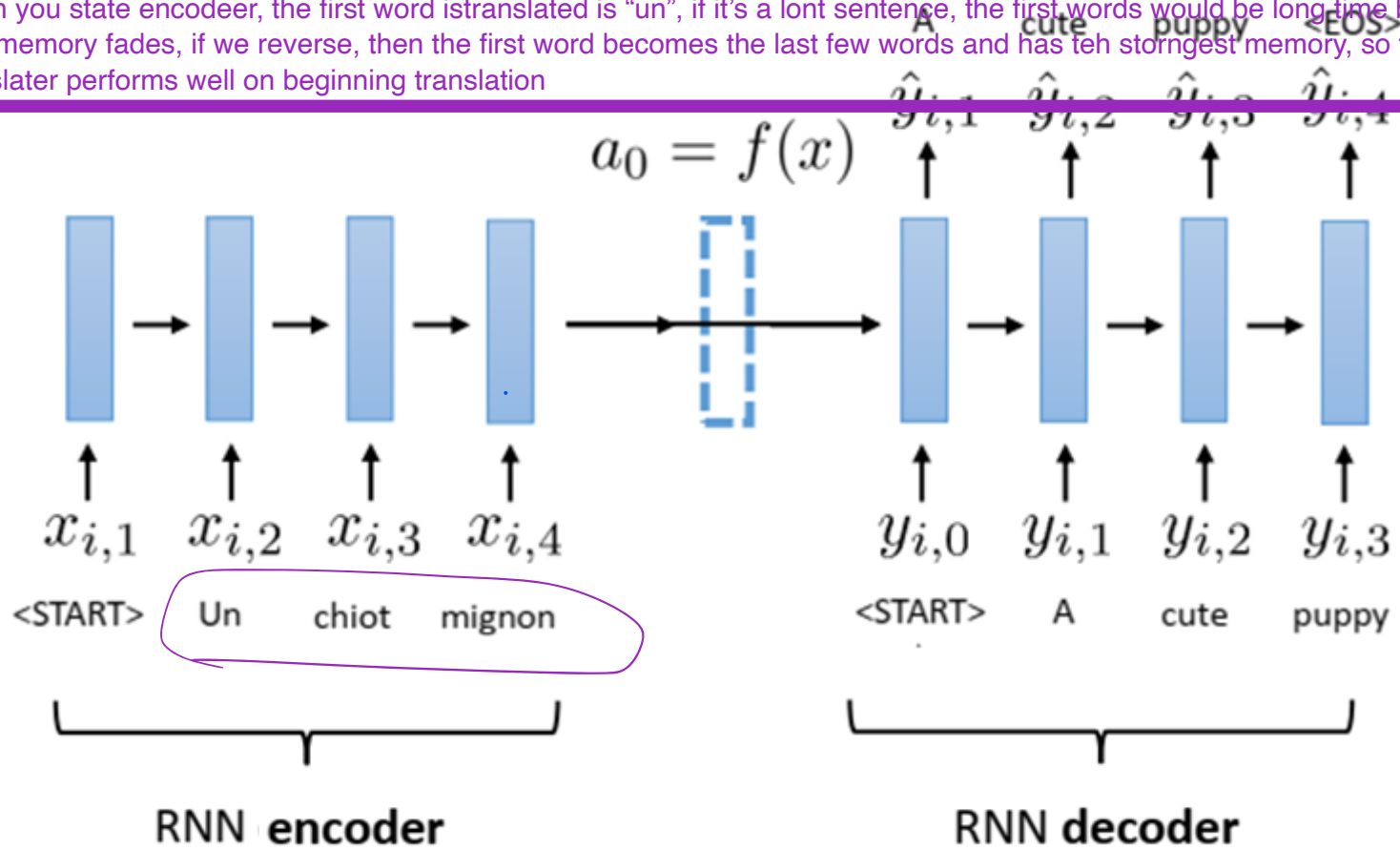
Recap: Image Captioning



Recap: Translation Model

sometimes the input for rnn encode is reversed, in below “....”, “mignon”, “chiot”, “un”
why?

when you start encoder, the first word is translated is “un”, if it’s a long sentence, the first words would be long time back and memory fades, if we reverse, then the first word becomes the last few words and has the strongest memory, so the translator performs well on beginning translation

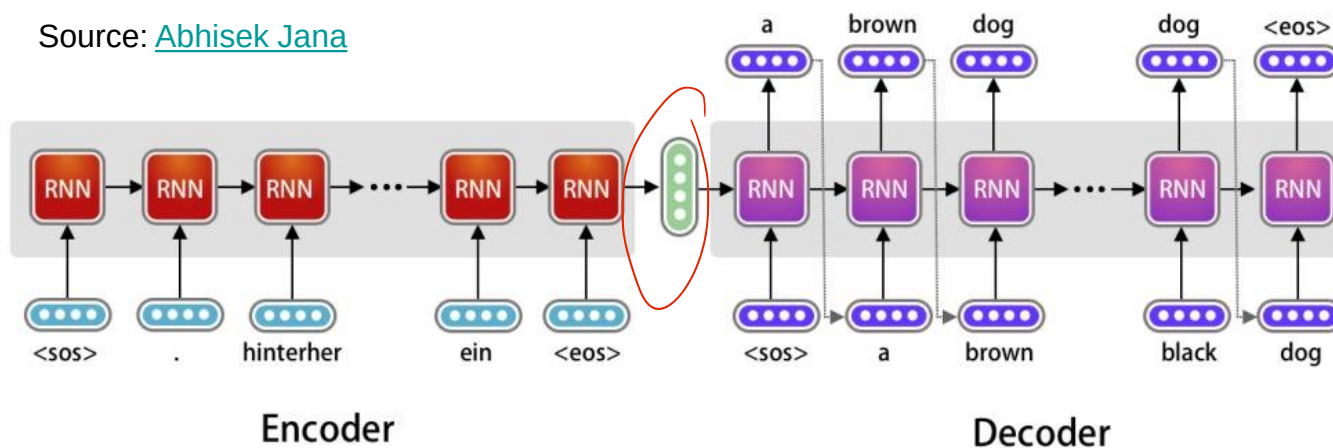


Sequence to Sequence Models

Today's Motivating Example

➤ Focus on **Neural Machine Translation**

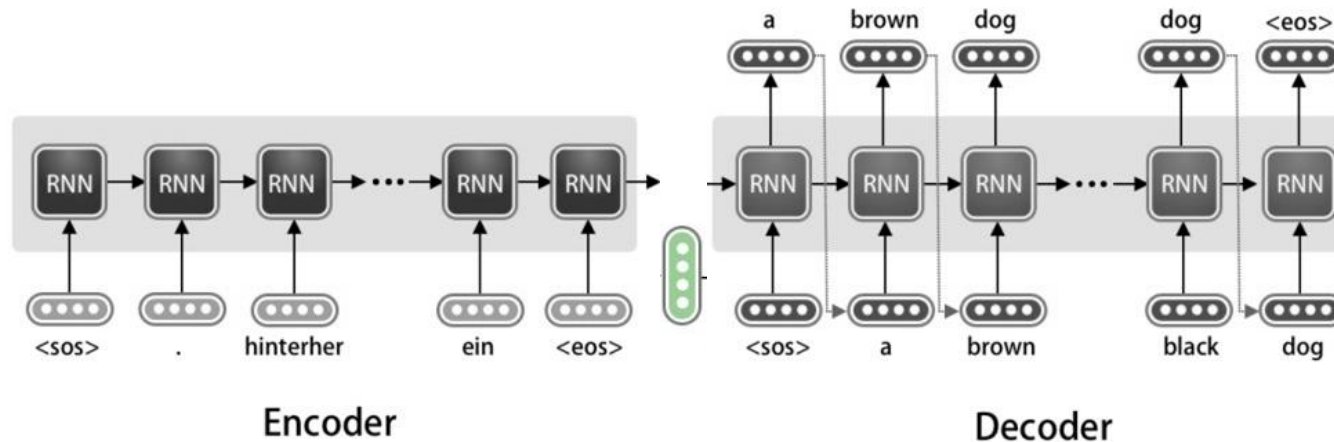
Source: [Abhisek Jana](#)



- Examine **encoder-decoder** networks
- Then improve upon the design using **attention** mechanisms ... and conclude with the **Transformer** architecture.

Challenges with long sequences

- Even using LSTMs, it is still **challenging to represent longer sequences of input data.**



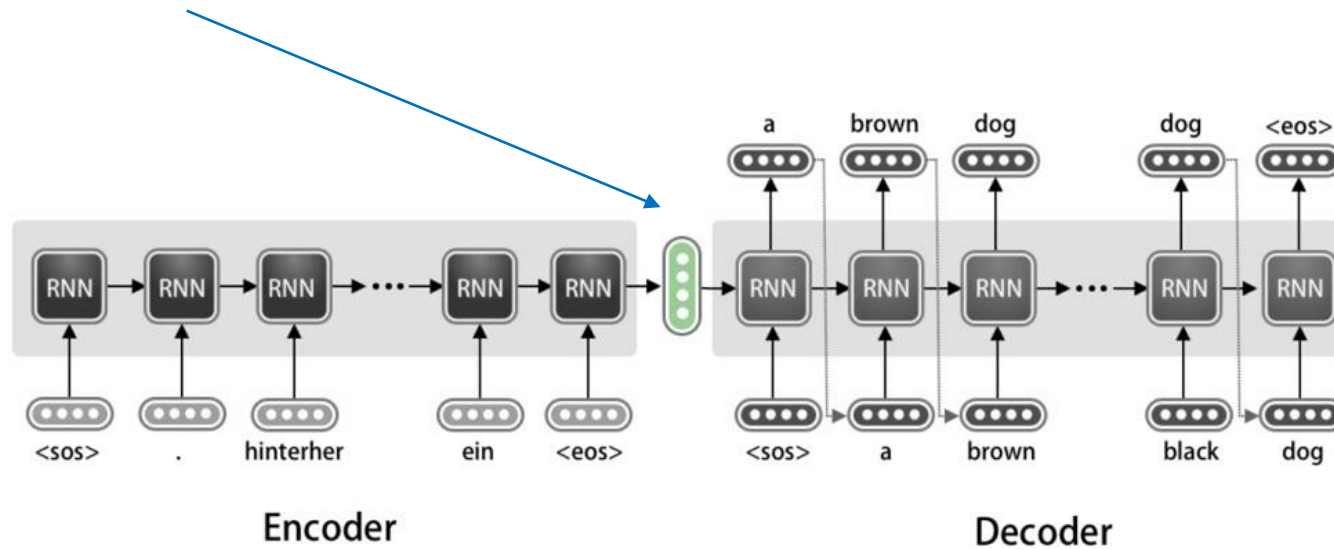
Q: Why is it difficult to remember long sequences?

Consider how information is passed to the decoder.

in encoder part, even we use skip connection, but since the sentence is too long, so not working well, then don't translate well
check class material in this slide

Challenges with long sequences

- Hard to summarize long sentence in a single vector.



- Helpful to allow the decoder to peek at inputs.

Q: How? Which input?

not only give a sentence vector/embedding, but also enables the embedding to get parsed information by each word in the sentence

check material slide

Attention

Attention

- When humans read or look:
 - They focus (attend) on some regions within the input
 - They pay less attention other regions that may not be important or relevant for decision making



Source: UX Matters

But How Does Attention Work?

- Q: How do we get a neural network to learn/use attention?
- Have the network learn an attention score
- Attention score indicates the importance of different parts of the input sequence
- We then **use the attention score to combine different parts of the input to create a “context” input**

give some fixed size of vectors to decode

Attention as Database Retrieval

- You can think of **attention** as being similar to **database retrieval**
- Standard Database Retrieval:
 - retrieves a **value** v_i for a **query** q based on a **key** k_i

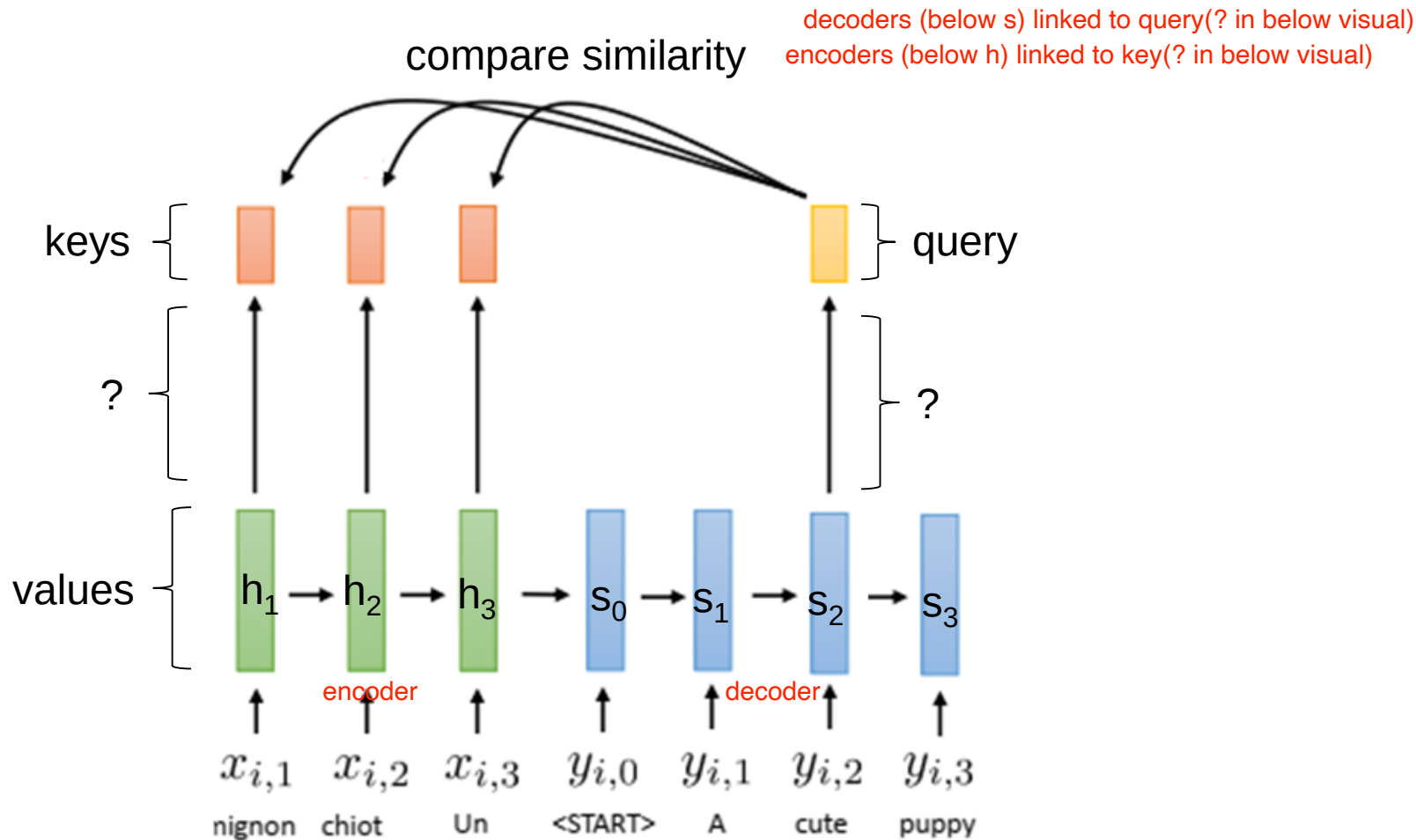
Attention as Database Retrieval

Attention-Based Retrieval:

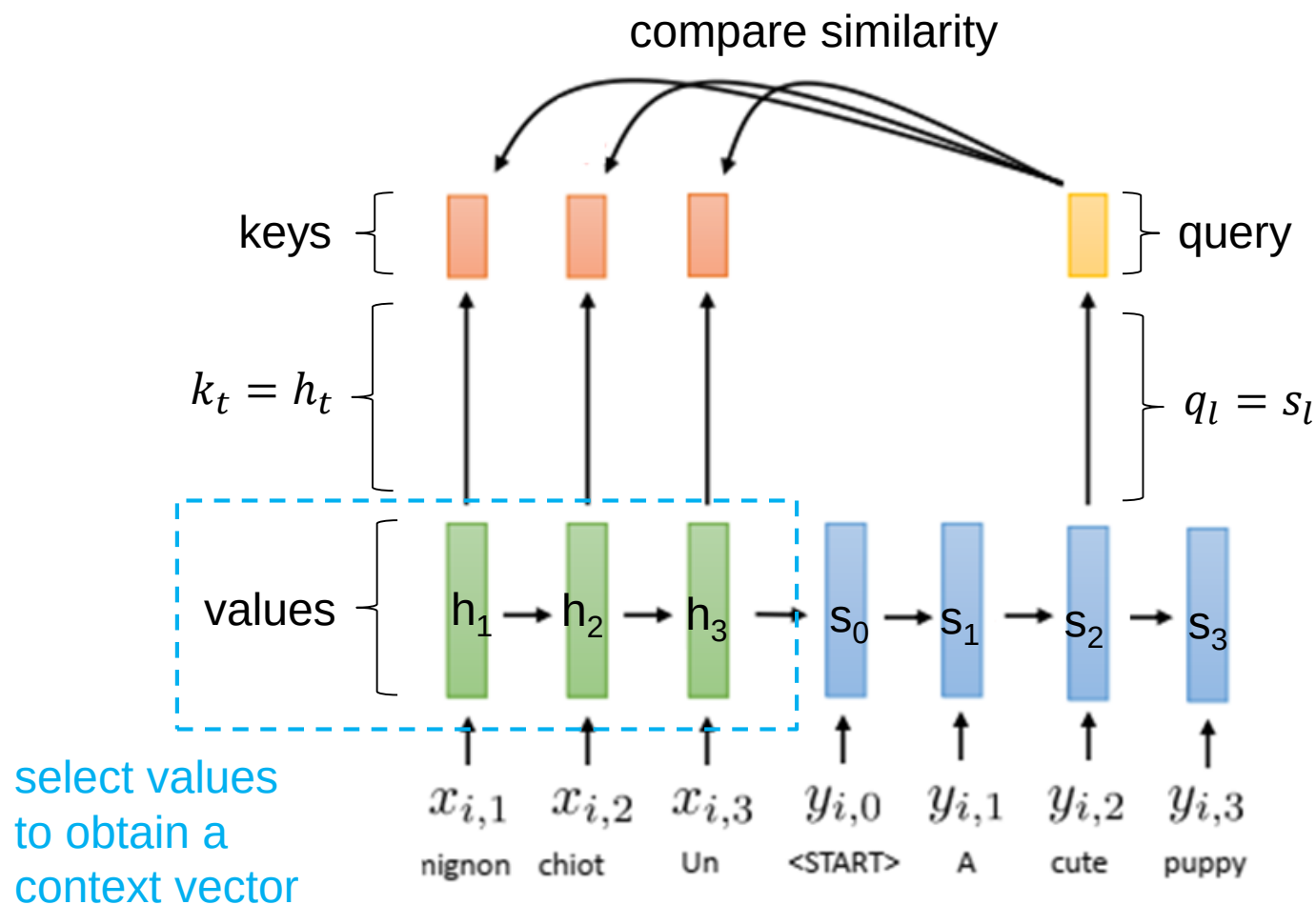
- rather than retrieving a single value for a query, it uses a **soft retrieval** what is soft retrieval??
- It retrieves all the values but then **computes their importance** w.r.t query based on the **similarity between the query and their keys**

$$\text{attention}(q, \mathbf{k}, \mathbf{v}) = \sum_i \text{similarity}(q, k_i) \times v_i$$

Constructing Attention Mechanism



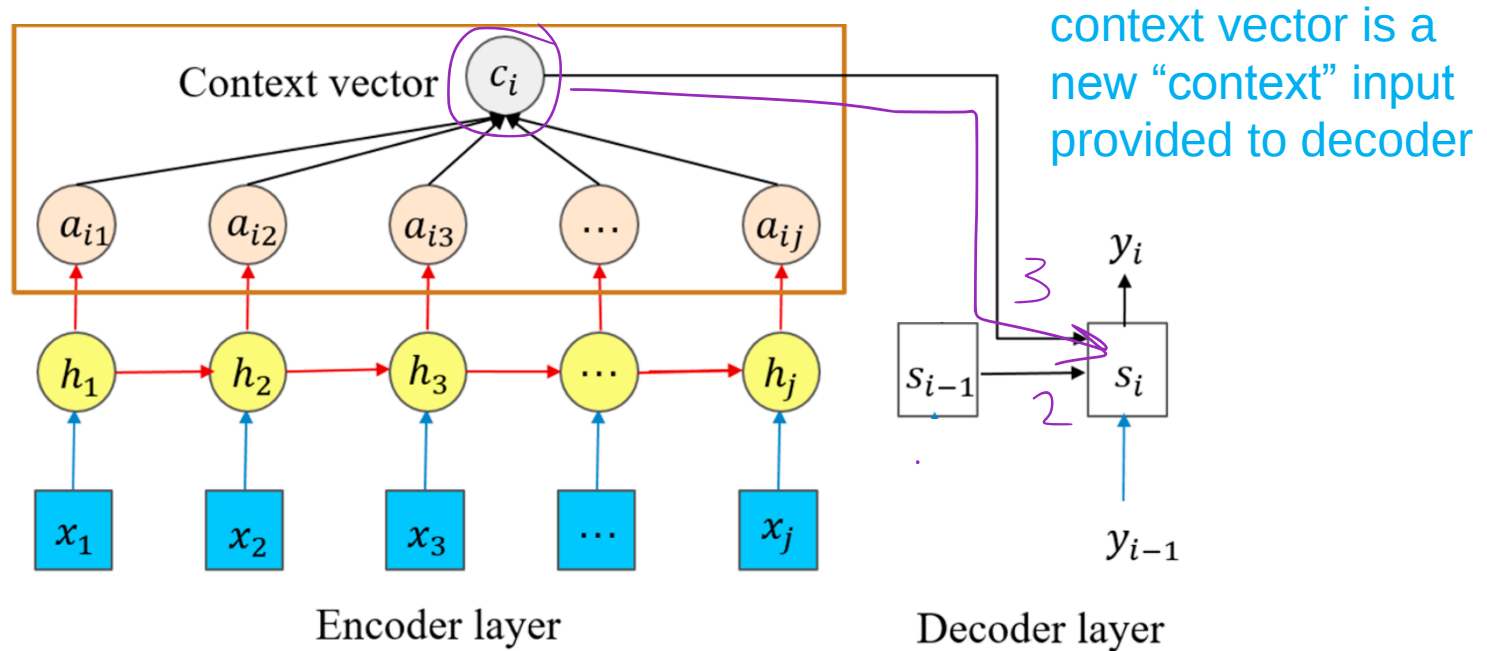
Constructing Attention Mechanism



$$v_t = h_t$$

Attention Mechanism

query 是decoder中要生成的第*i*个词。同时看decoder中*i*-1的 hidden state和所有encoder hidden state, 计算alignment score (就是相似度), 通俗理解就是, 下一个生成的词和哪个input最相关。相似度的计算用到q和k, $k=v$ 就是encoder hidden state。接着, 计算 $\text{softmax}(\text{alignment score})$ 就是attention score, 得到一个针对encoder hidden state weight vector。用这个权重向量对v做加权平均, 得到context vector。通俗理解就是, 为了生成第*i*个词 脑子现在汇总的上下文信息



Step 1

alignment score

$$a_{ij} = f(s_{i-1}, h_j)$$

k

Step 2

attention score

$$\alpha_{ij} = \text{softmax}(a_{ij})$$

$$= \frac{\exp(a_{ij})}{\sum_j \exp(a_{ij})}$$

Step 3

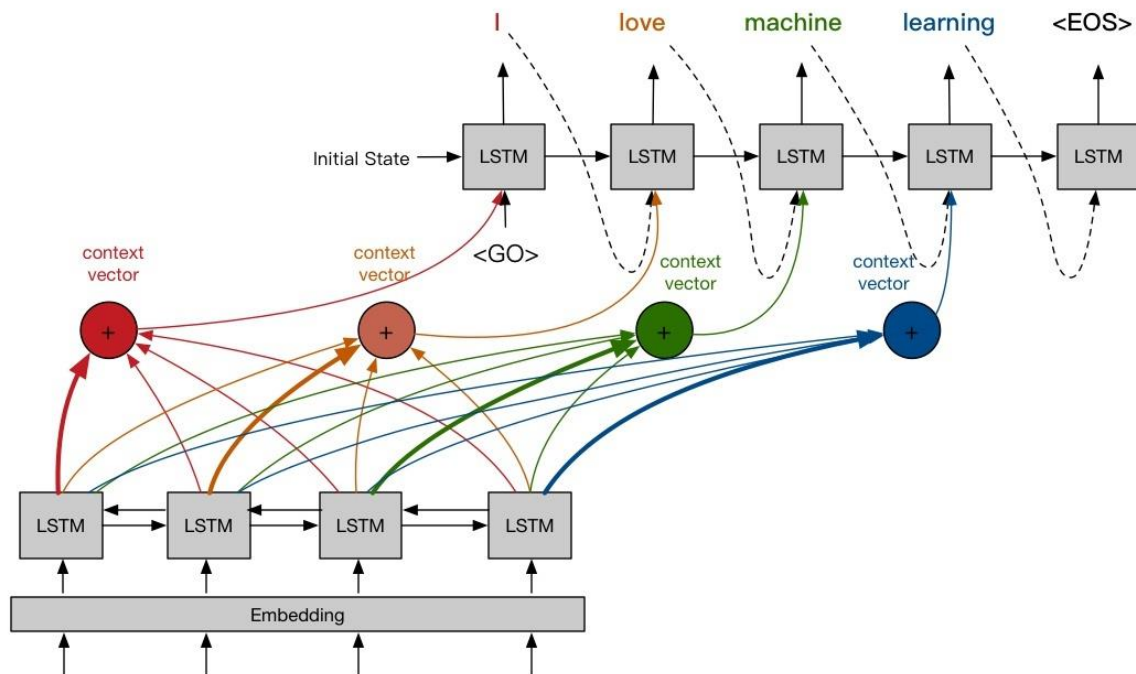
k and v are the same so here h_j is the v

$$c_i = \sum_j \alpha_{ij} h_j$$

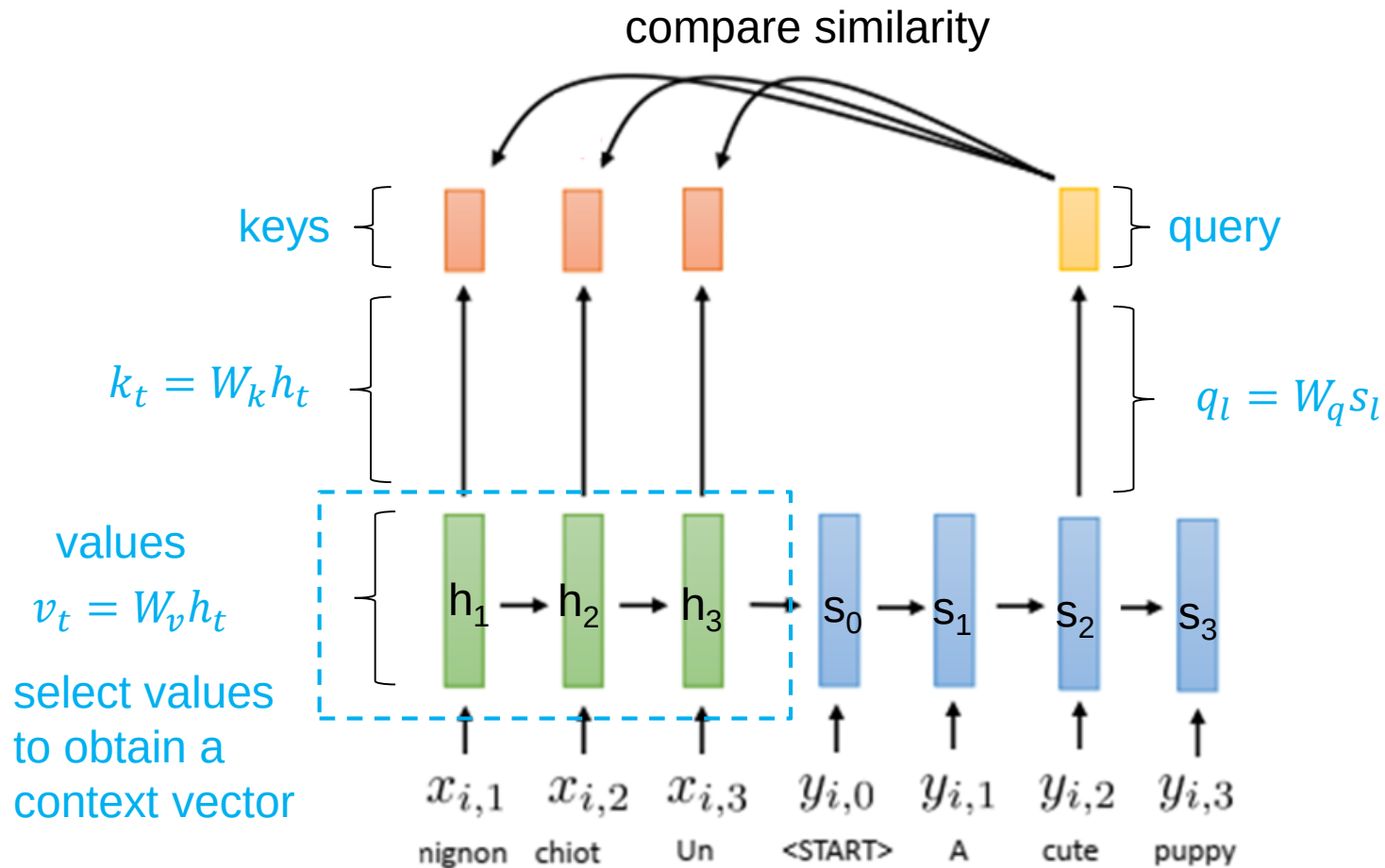
get the fixed vector c_i and feed it into the next step

Summary: Attention Mechanism

- Instead of just sending the encoder's final hidden state to the decoder, it gives access to all the encoder's past hidden states.



Attention Mechanism (Variant)

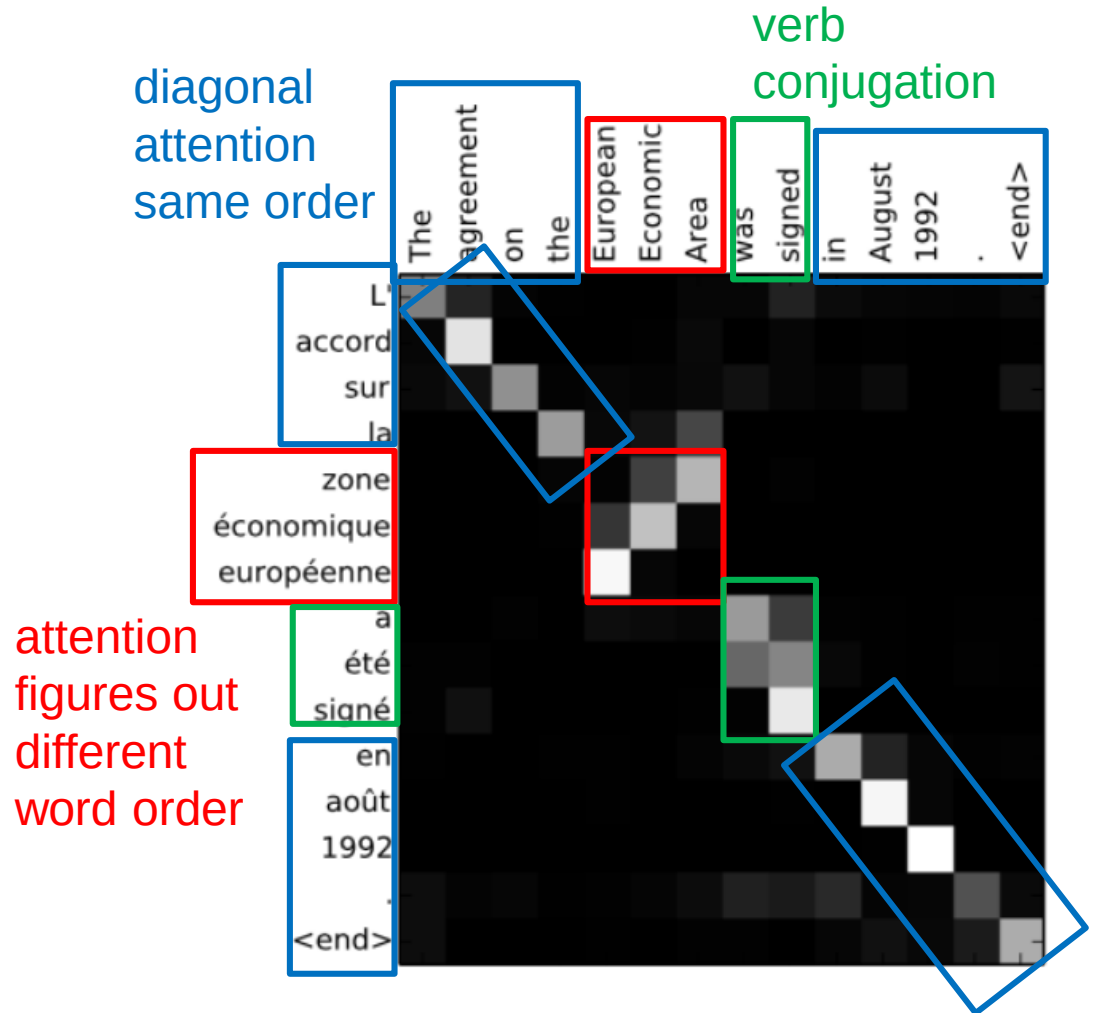


Visualization of Attention Weights

English to French translation:

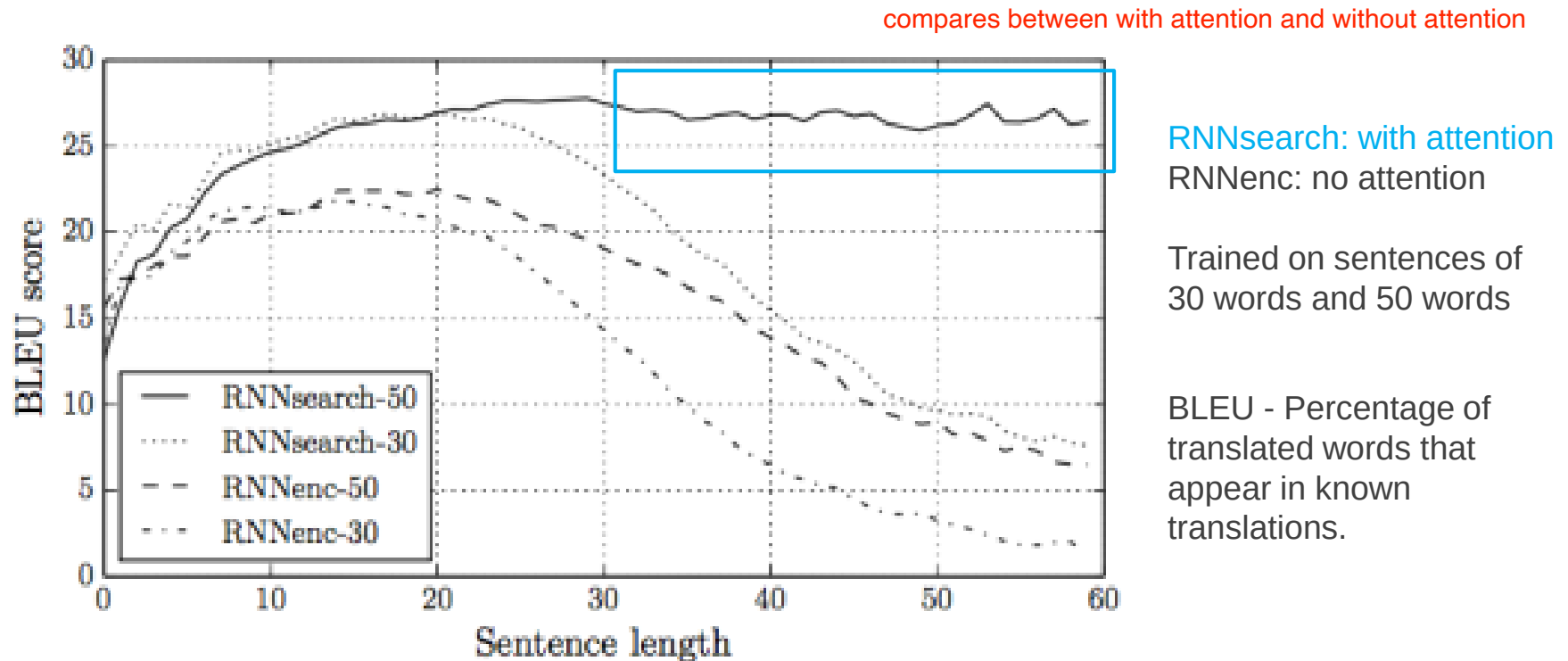
Input: "The agreement on the
European Economic Area
was signed in August 1992."

Output: "L'accord sur la zone
économique européenne a
été signé en août 1992."



Source: Bahdanau et al, ICLR 2015

Attention Performance



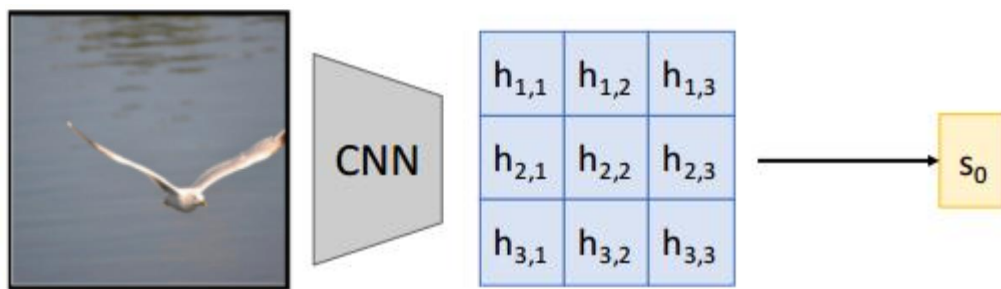
The attention-based translation model does much better than model without attention on long sentences.

CNN + RNN + Attention

图像描述 + 注意力:

CNN 把图片切成一堆局部特征, RNN 在生成每个单词时, 通过注意力从这些局部特征里“挑重点”合成一个上下文向量, 然后用 context vector + 前一个词, 一步步把句子说完

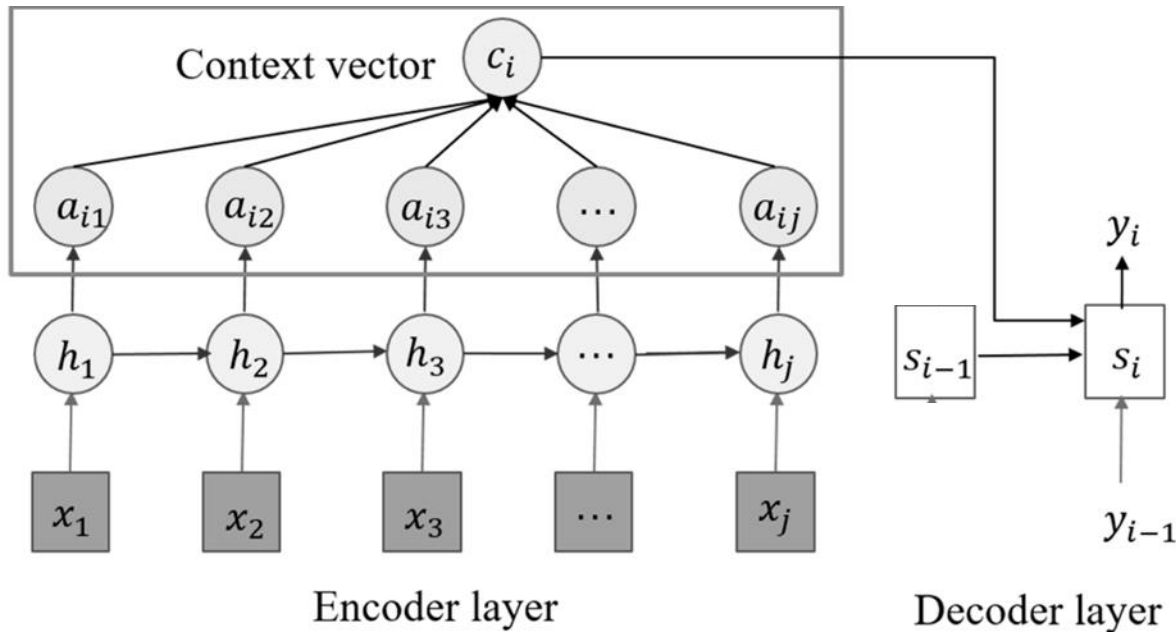
Image Captioning with Attention



Use a CNN to compute a grid of features for an image

Can use attention on any set of input vectors $\{h_j\}$!

Note: Unordered Set of Information



Notice that the decoder treats the input information as an unordered set $\{h_j\}$

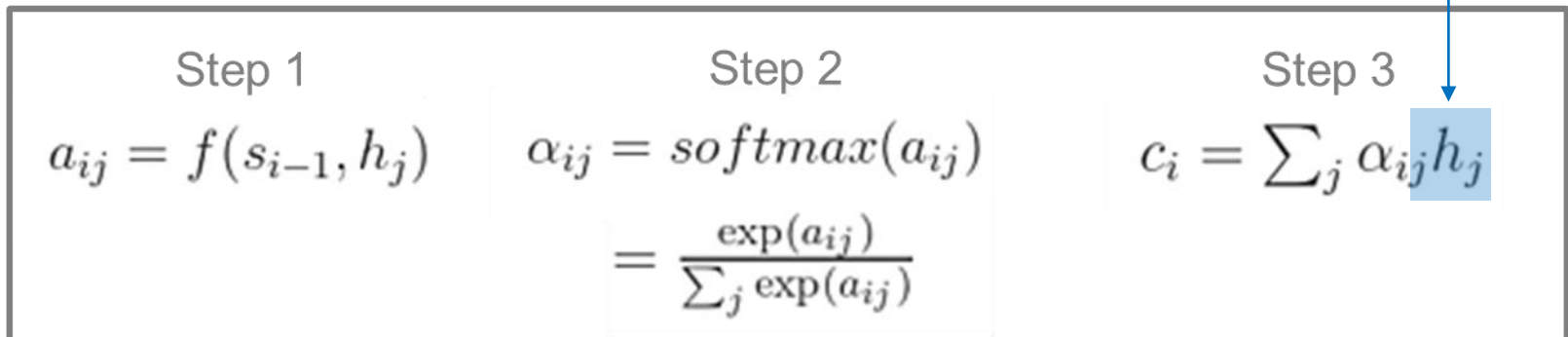


Image Captioning with Attention

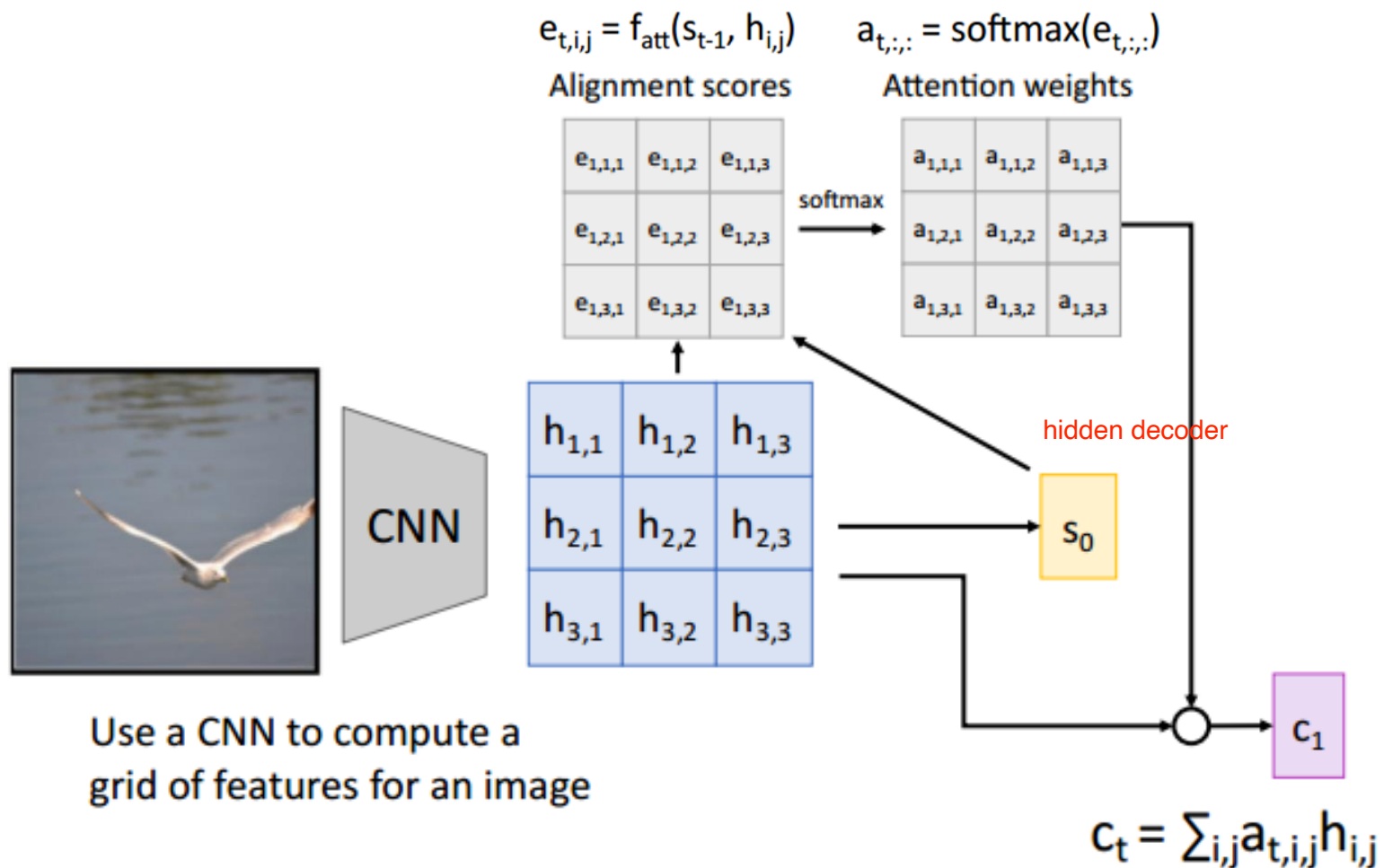


Image Captioning with Attention

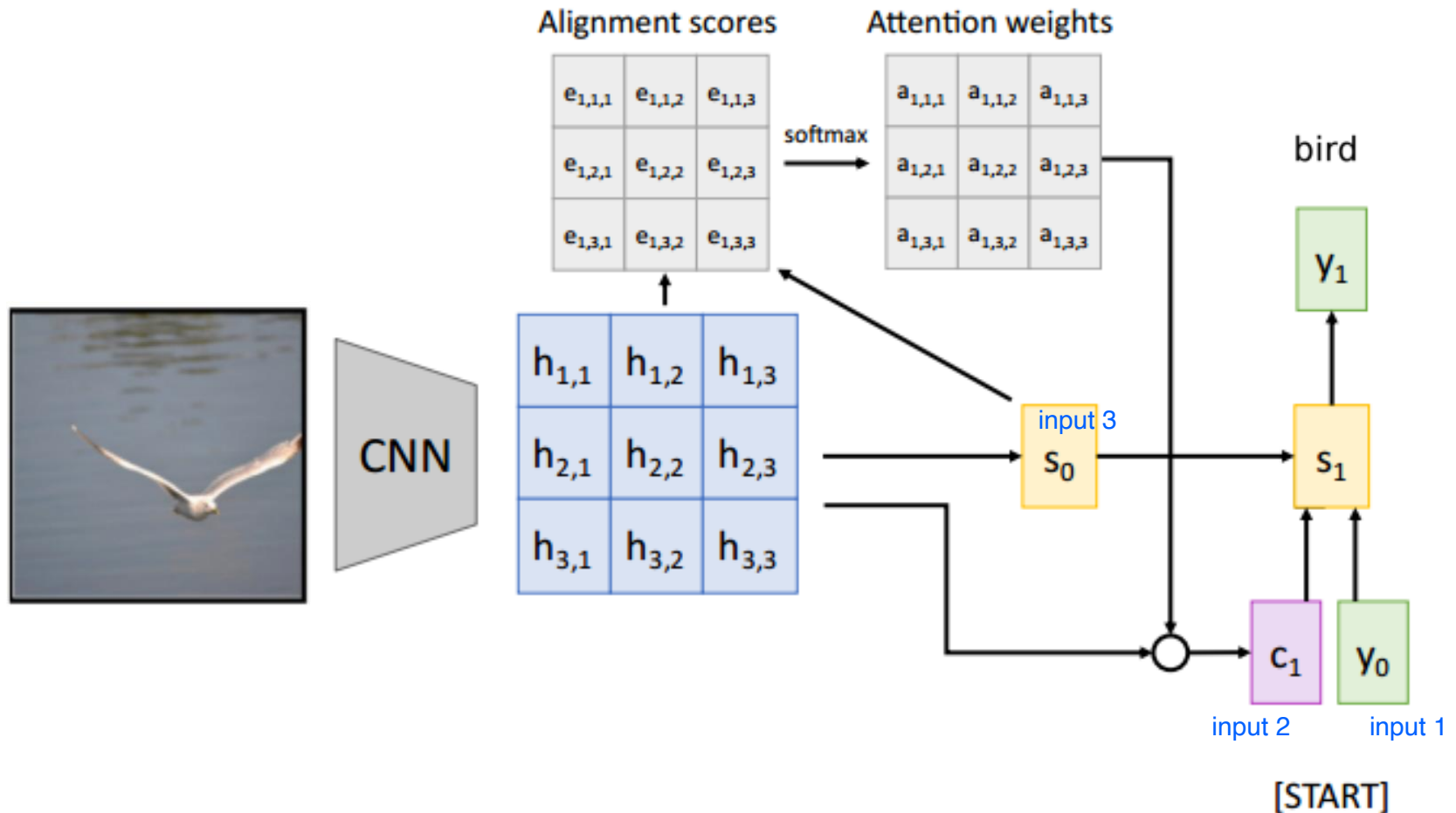


Image Captioning with Attention

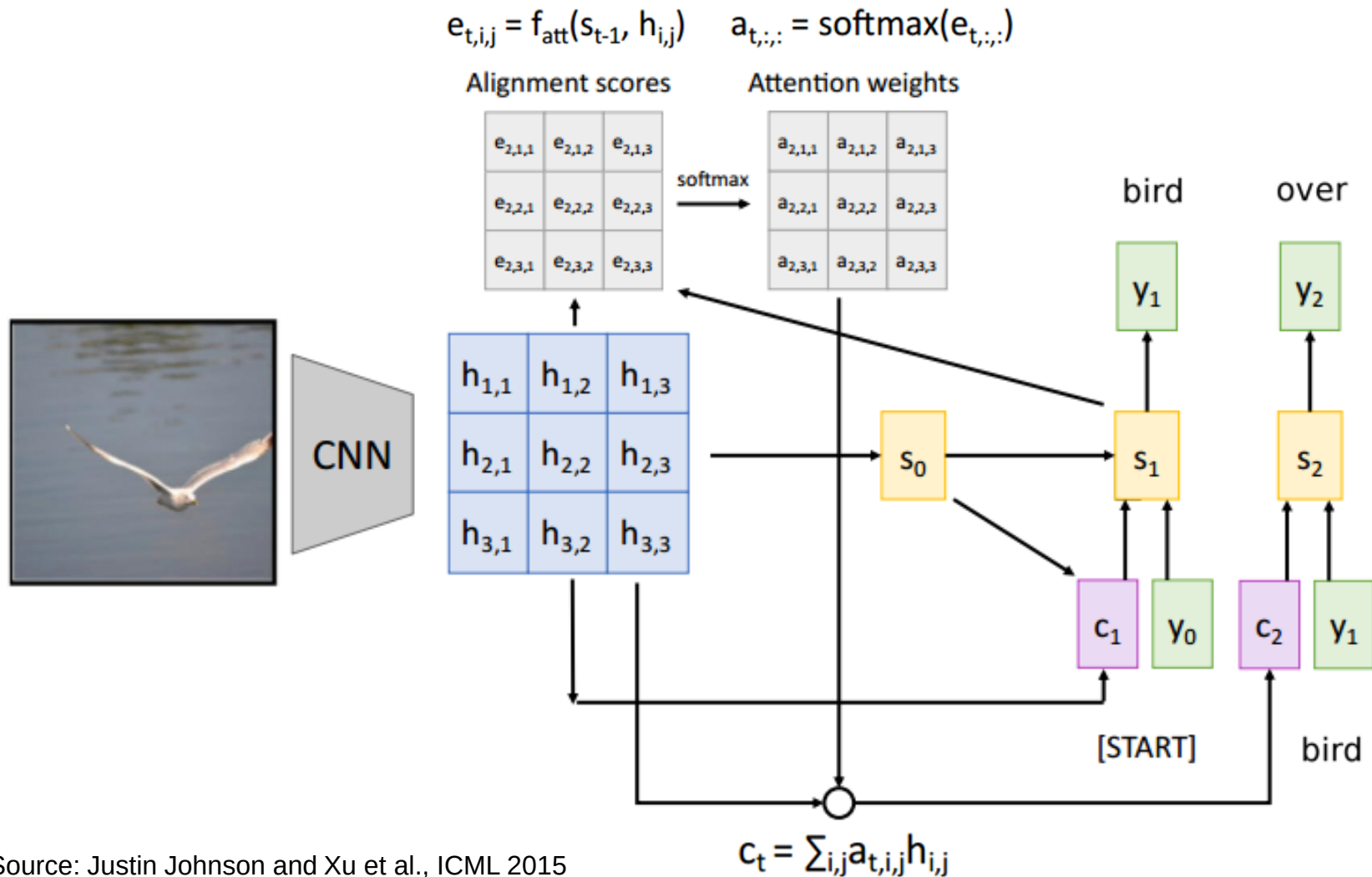


Image Captioning with Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

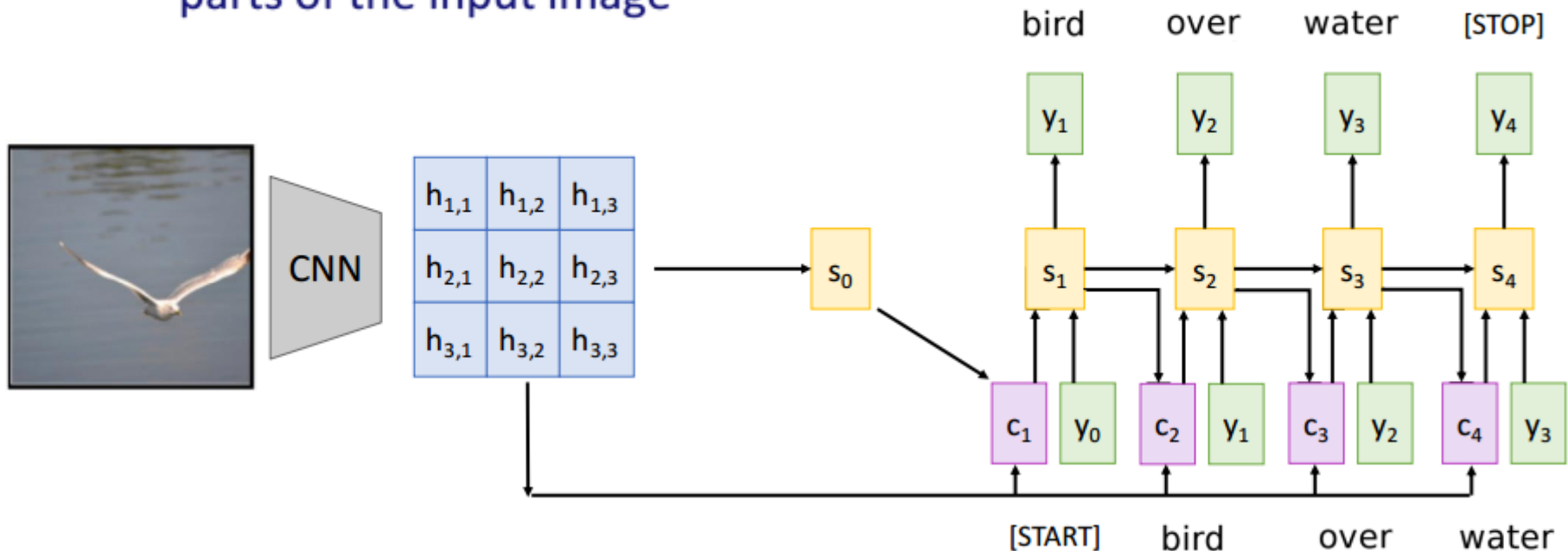


Image Captioning with Attention



Attention weights at each timestep kind of like saccades of human eye



The **fovea** is a tiny region of the retina that can see with high acuity

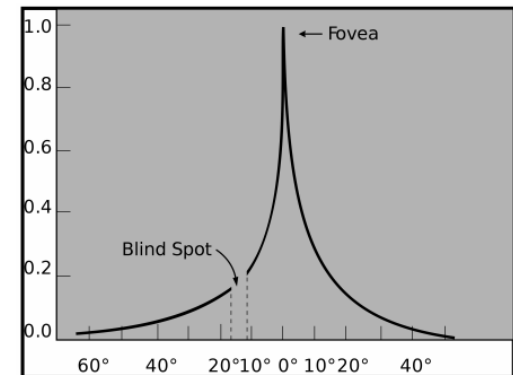


Image Captioning with Attention



A dog is standing on a hardwood floor.



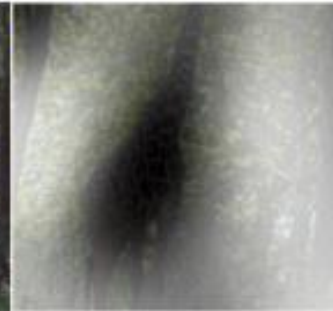
A stop sign is on a road with a mountain in the background.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.



Transformers

Optimization Problems

➤ So far, we have only been working with **recurrent neural networks...**

➤ Q: What is a common limitation among all variants of RNNs?

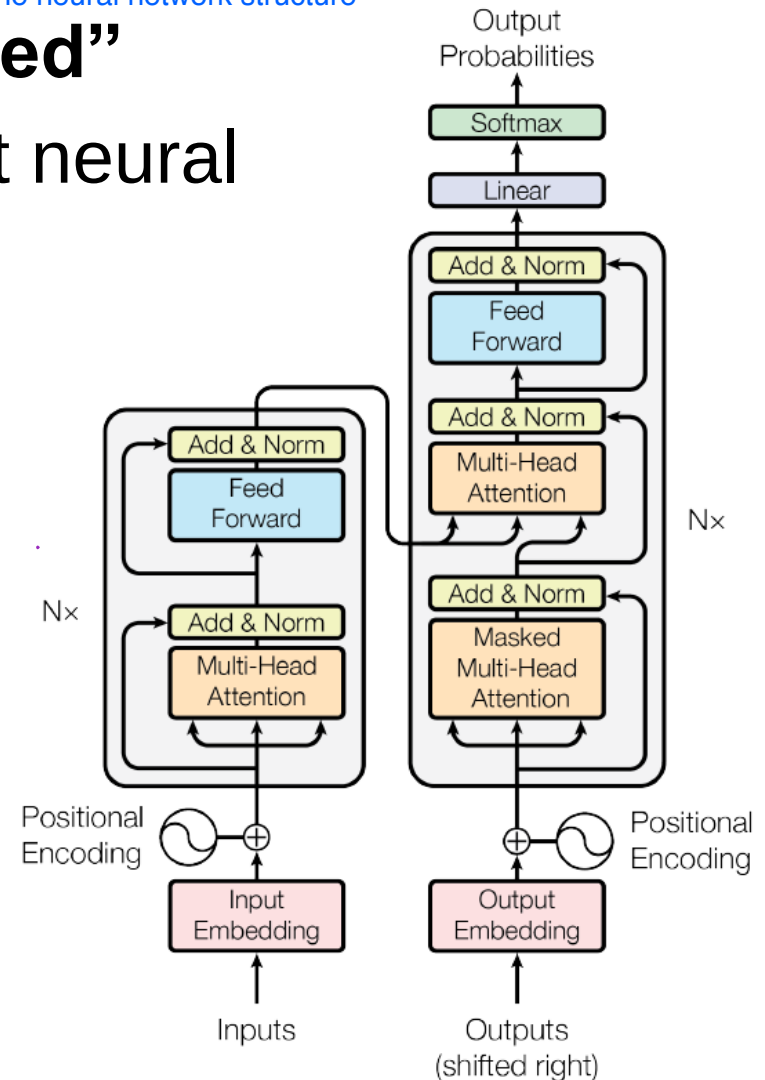
it's a serial processing --> SLOW!!

parallel processing can solve and speed up

Transformers

still follow the neural network structure

- **“Attention Is All You Need”**
- Does away with recurrent neural networks
- **input is the entire sequence** (e.g., word sequence, can also be other sequences)
- **performs parallel operations**, works well on GPU



Attention is all you need

- Q: How can we process sequences without the recurrent (serial) connections?
- A: We require a new type of attention mechanism that can work in parallel.

Self-Attention

Attention mechanism can be defined:

- Between two sequences

mix the attention of the encode and decoder
encoder focuses on k, v
decoder focused on q

- can think of it as cross-attention

- e.g., encoder and decoder in machine translation

- Or on a single sequence

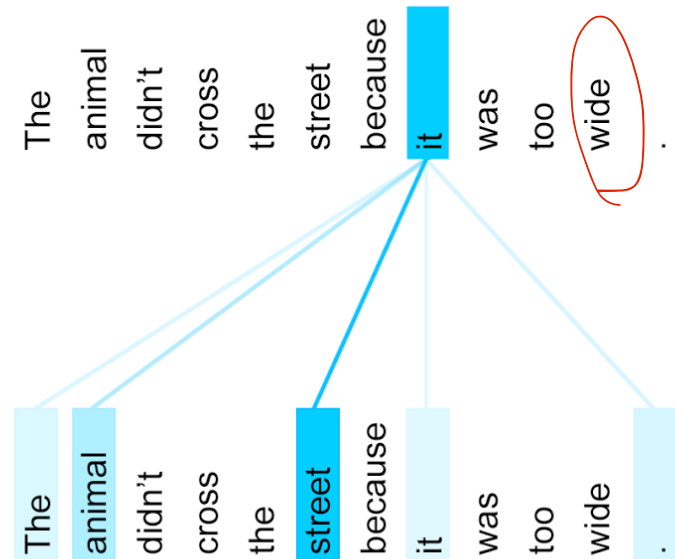
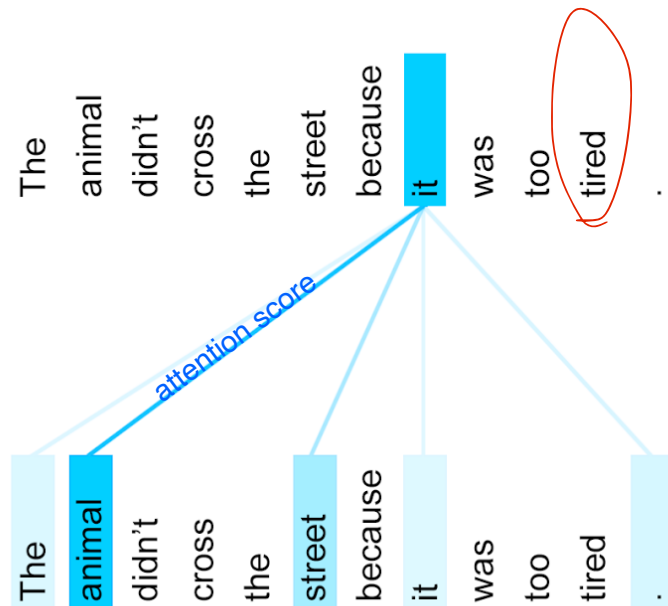
all of k, v, q

- referred to as self-attention

- compute a representation of the same sequence

Self-Attention

Compute attention of input with respect to itself: for a given token of the input, compute attention weight for all other tokens in the sequence.



the attention enables the machine to learn what "it" refers to

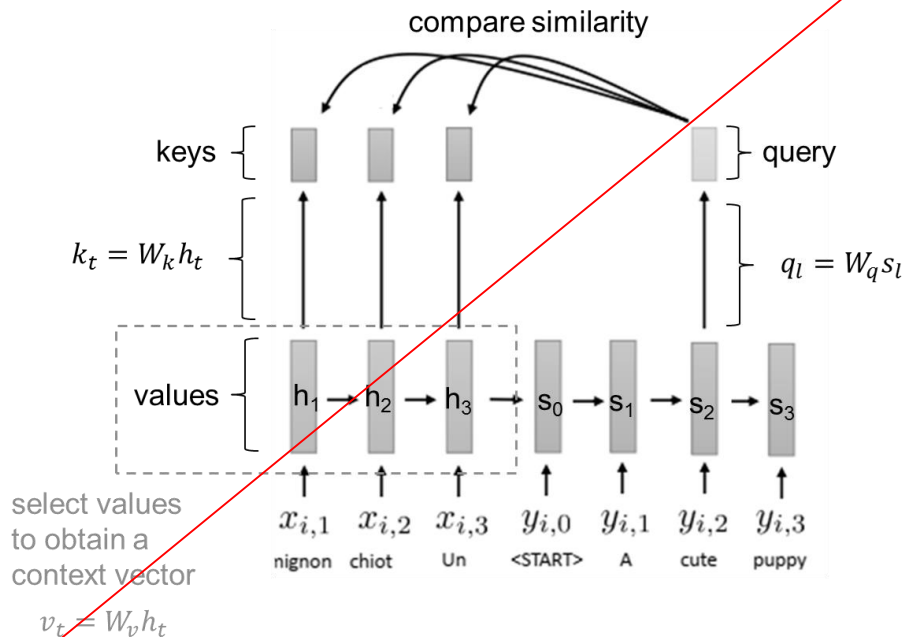
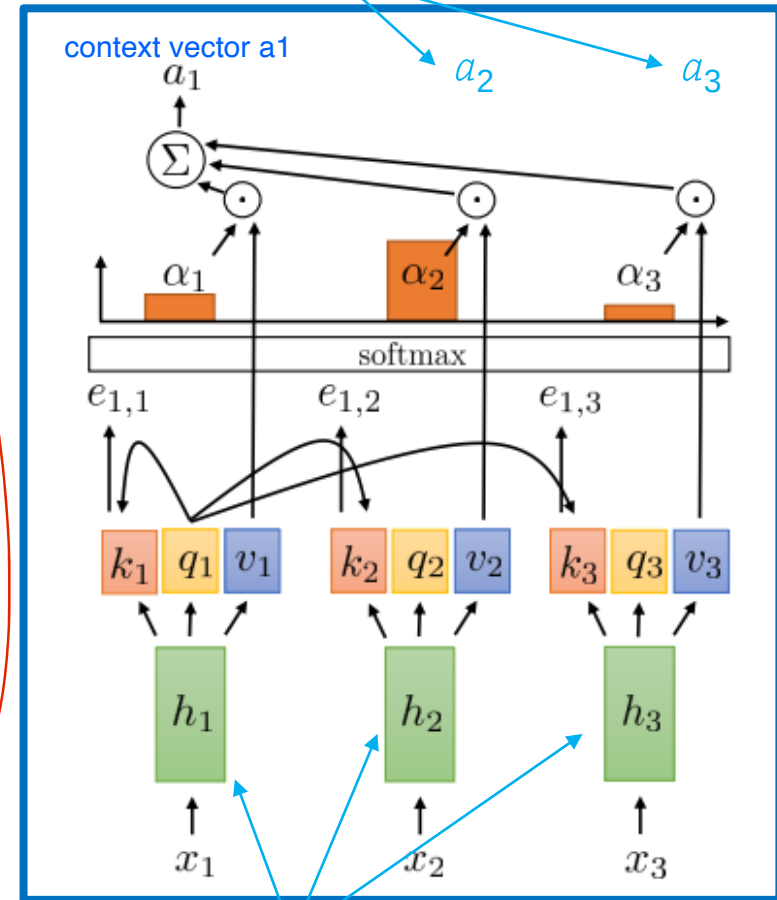
self-attention has learned "it" could refer to different entities in different contexts.

Self-Attention

link q1 to all other keys to know how similar it to other queries, then it gets the alignment score, then softmax, then get the attention score for q1. Similar to other q, then we get the attention scores for all queries, sum up, then get the context vector a1

Previously, used cross-attention, now we will use self-attention

similar process to obtain a_2 and a_3



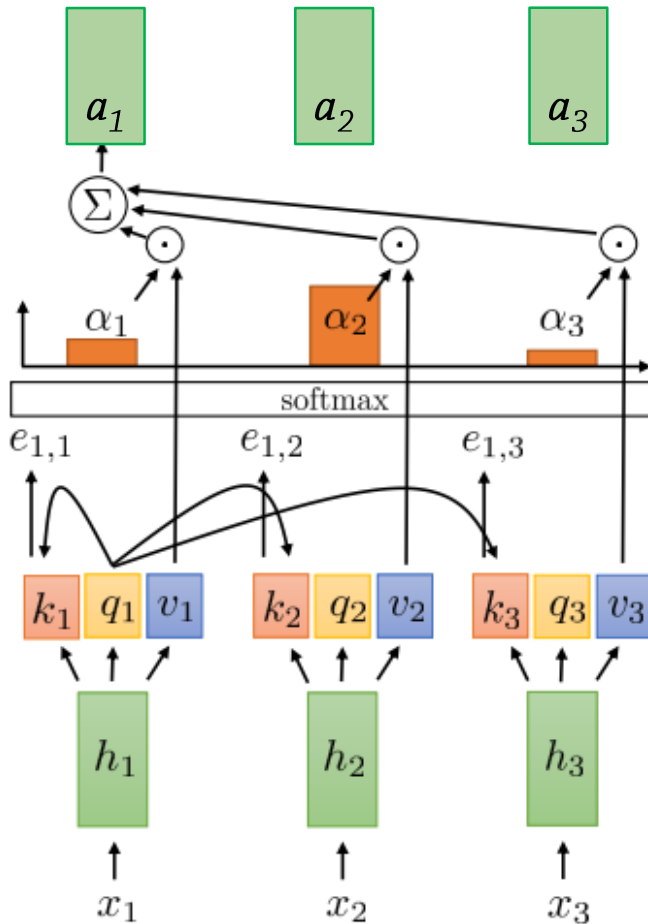
this is serial processing, not use anymore

Source: Sergey Levine

like hidden states – but without the recurrence

Self-Attention

input are the 3 vectors h_1-h_3 , after processing, got 3 output vectors a_1-a_3



$$a_l = \sum_t \alpha_{l,t} v_t$$

Step 3

$$\alpha_{l,t} = \text{softmax}(e_{l,t})$$

Step 2

$$e_{l,t} = q_l k_t$$

this is the cosine similarity
to know how close the queries and keys

Step 1

$$k_t = W_k h_t$$

$$q_t = W_q h_t$$

$$v_t = W_v h_t$$

Step 0

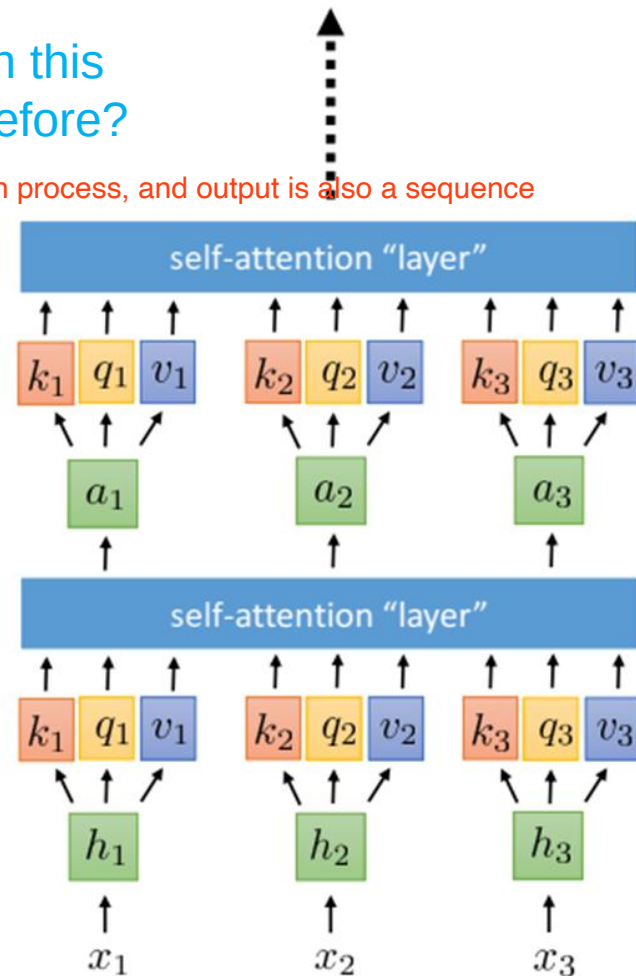
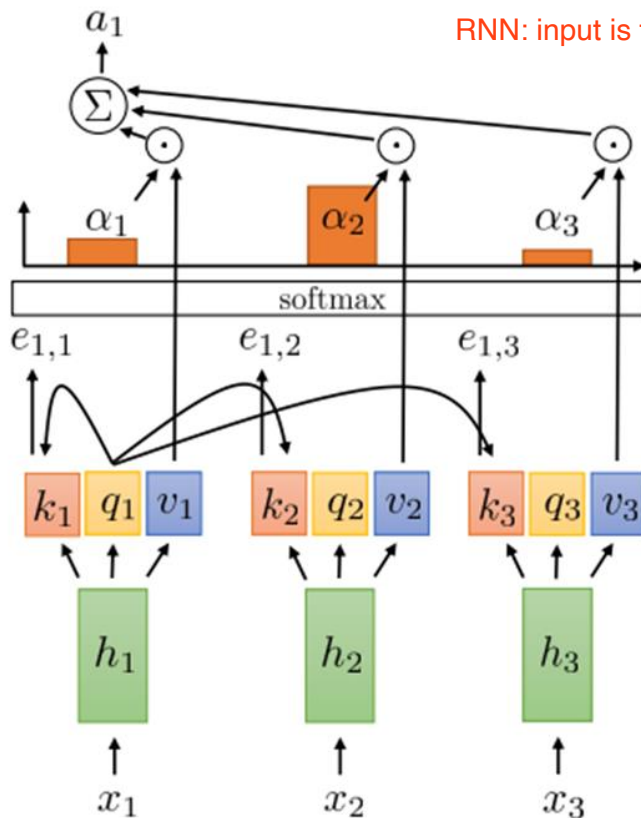
inputs are very generalized, could be unordered tokens, any kinds of inputs, doesn't have to be embeddings

Source: Sergey Levine

Stacked Self-Attention Layers

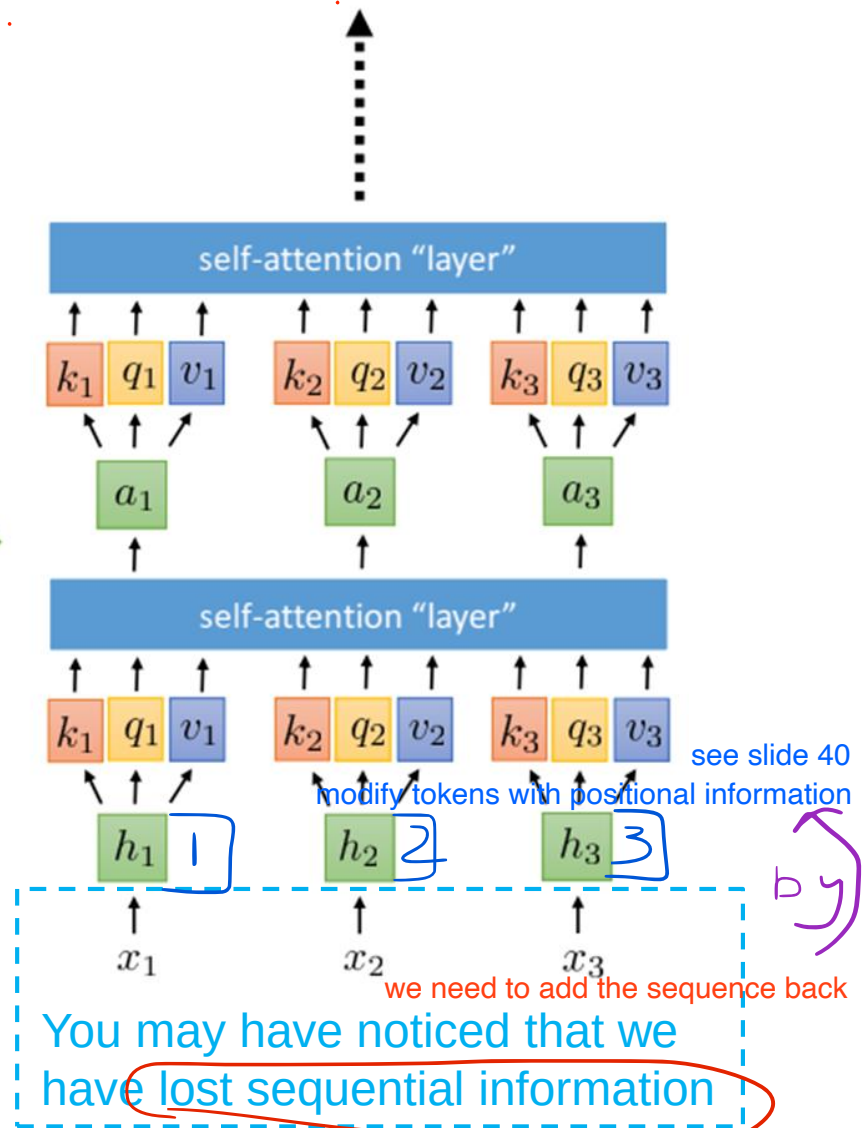
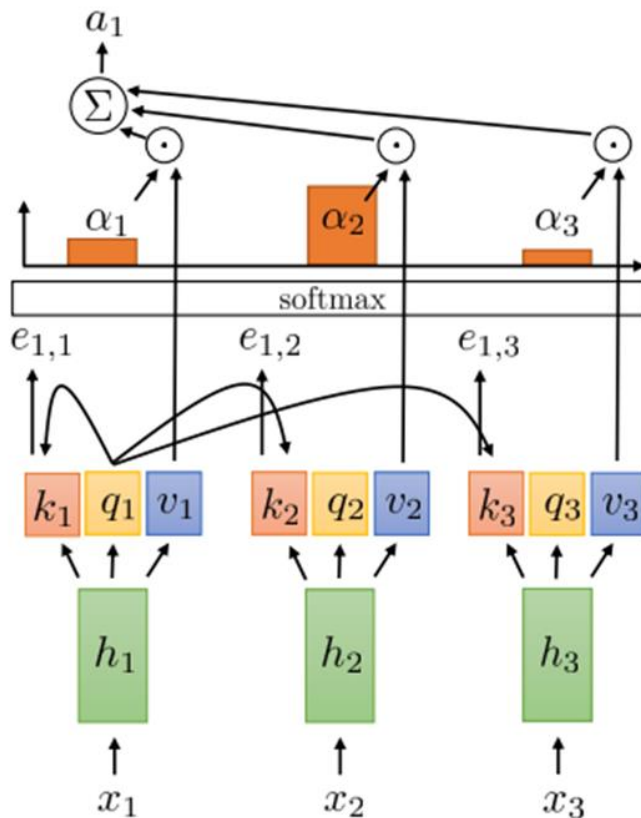
Q: Where have we seen this stacked arrangement before?

RNN: input is the sequence, then process, and output is also a sequence



Self-Attention

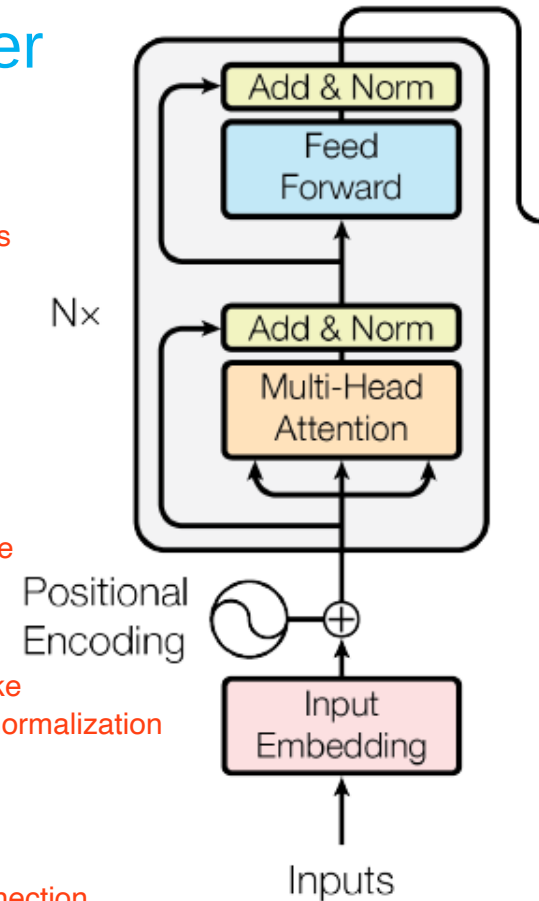
Attention + Positional Encoding => Transformer Encoder



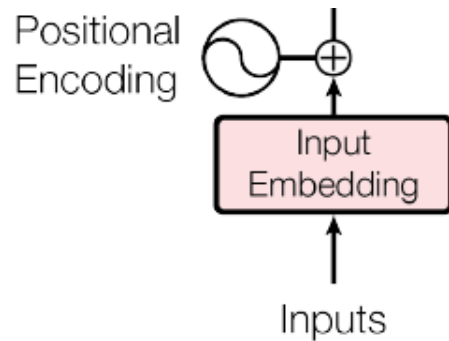
Transformer: Encoder

Encoding position, is just one of several tricks used in the transformer architecture

1. Inputs (Tokenization) tokenization: text to numbers
2. Positional Encoding
3. Multi-Head Attention (Self Attention) like kernel in CNN, just here is for sequential data
4. Normalization help make gradients stable: like batch normalization, z-score normalization
5. Feed Forward MCP for non-linearity
6. Residual Connections =skip connection



Positional Encoding -1

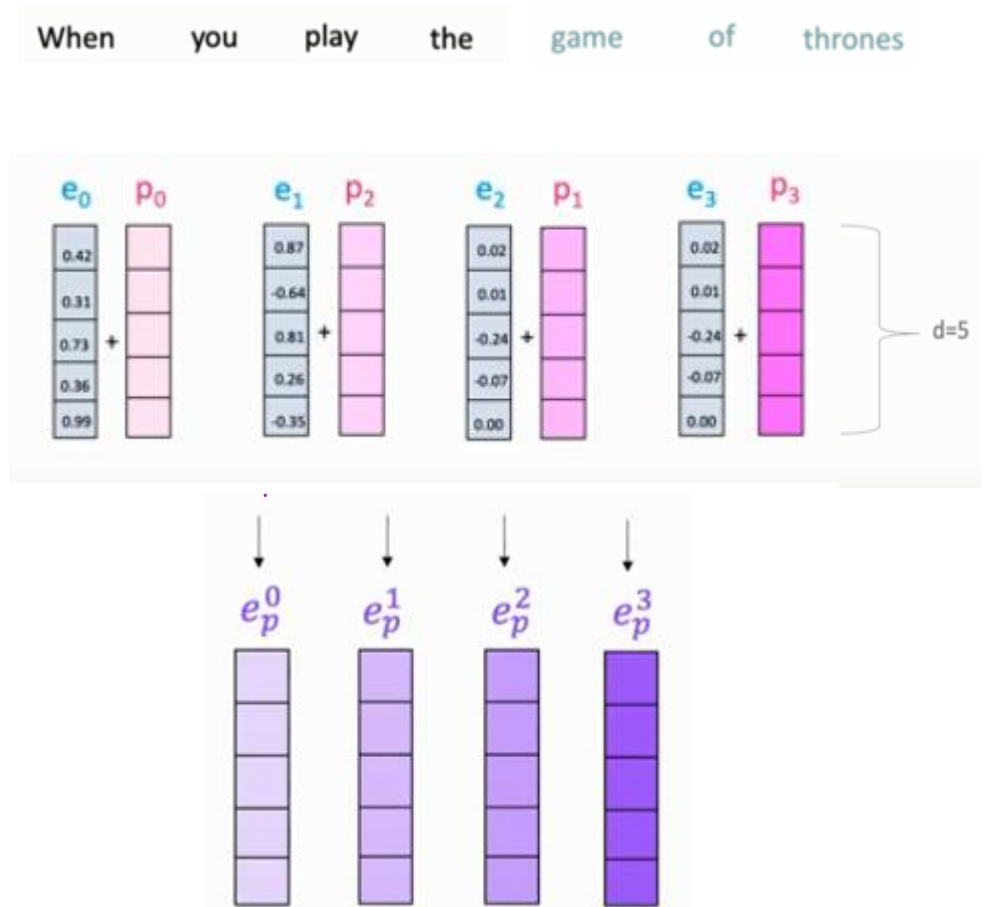


- Applies an embedding to distinguish each position

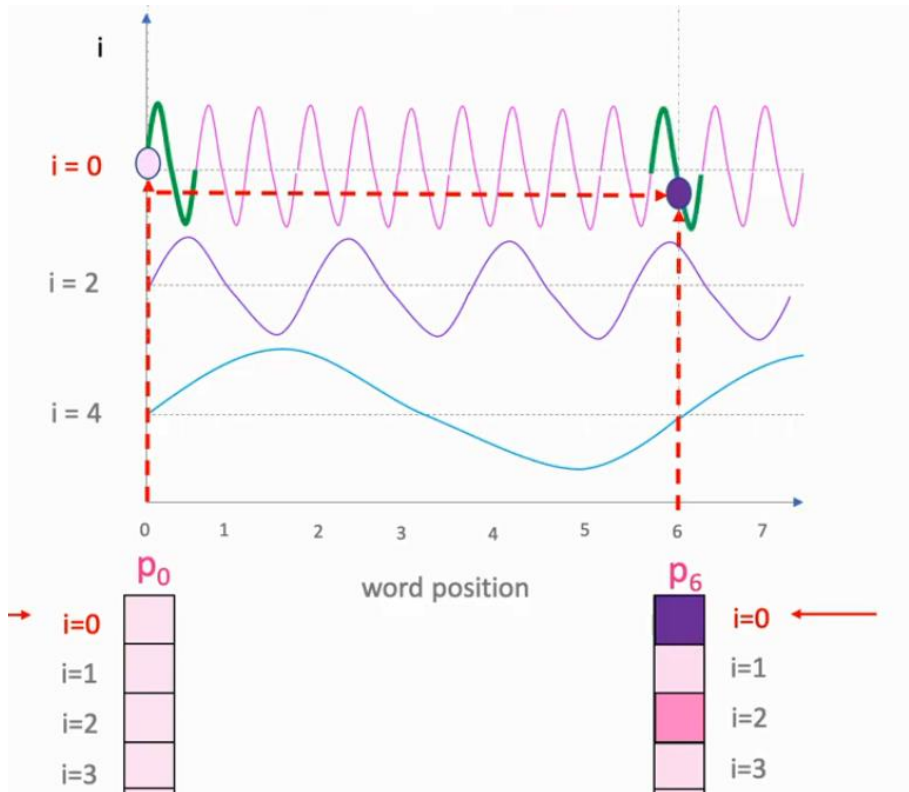
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

d: length of the vector



Positional Encoding -2



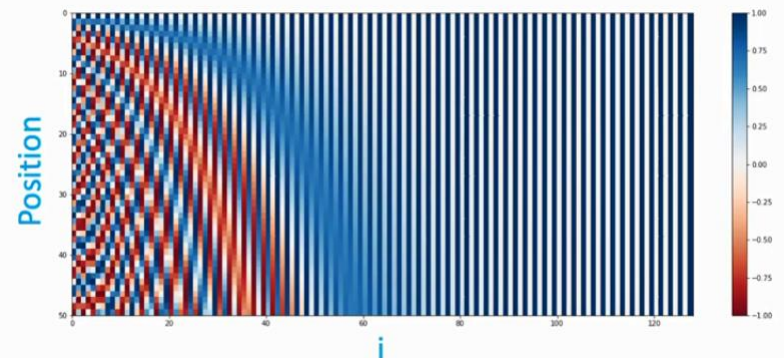
1	2	3	4
e_0	e_1	p_0	e_3
0.42	0.82	0.02	0.38
0.31	0.79	0.01	0.16
0.73	0.76	-0.24	0.01
0.36	0.74	-0.07	0.09
0.99	0.71	0.00	0.00
		0.27	-0.78
		0.39	-0.87
		0.50	-0.94
		0.60	-0.98
		0.69	-1.00

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

odd positions

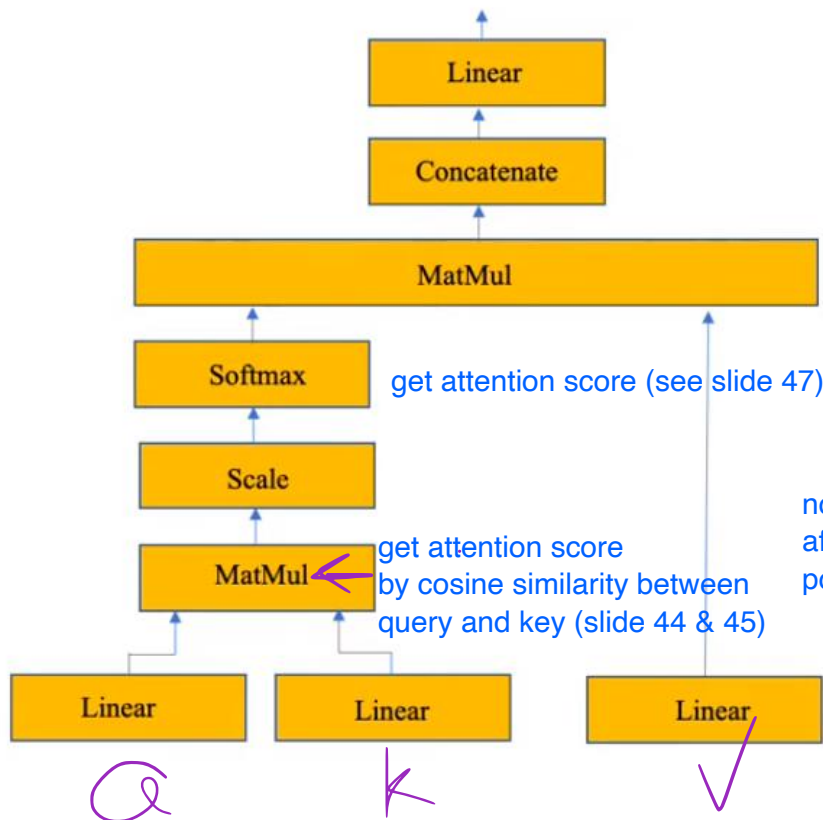
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

even positions

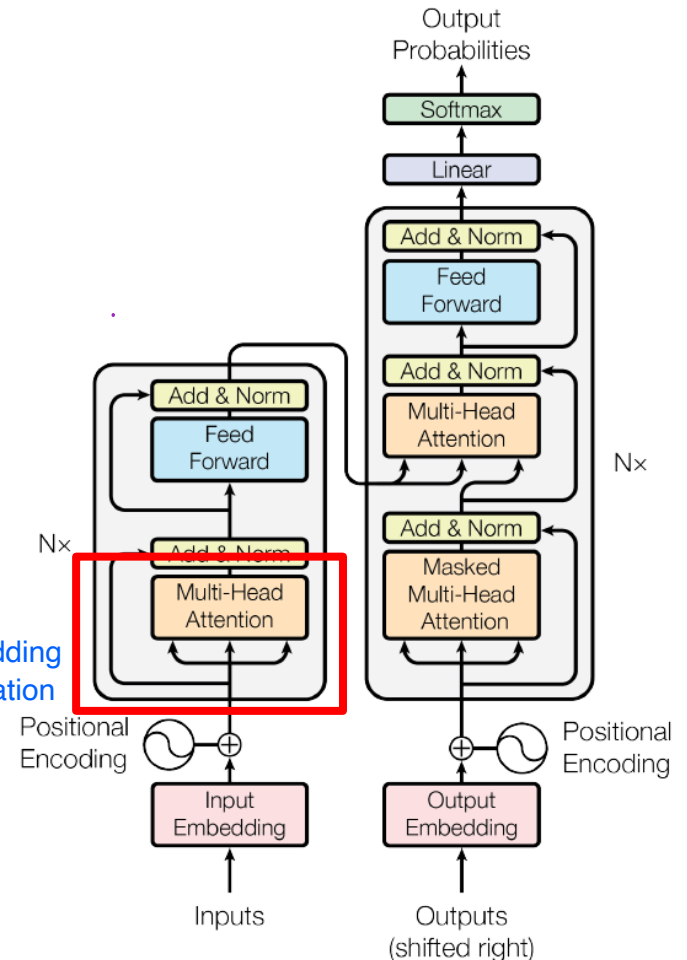


Source: Heduo - Math of Intelligence

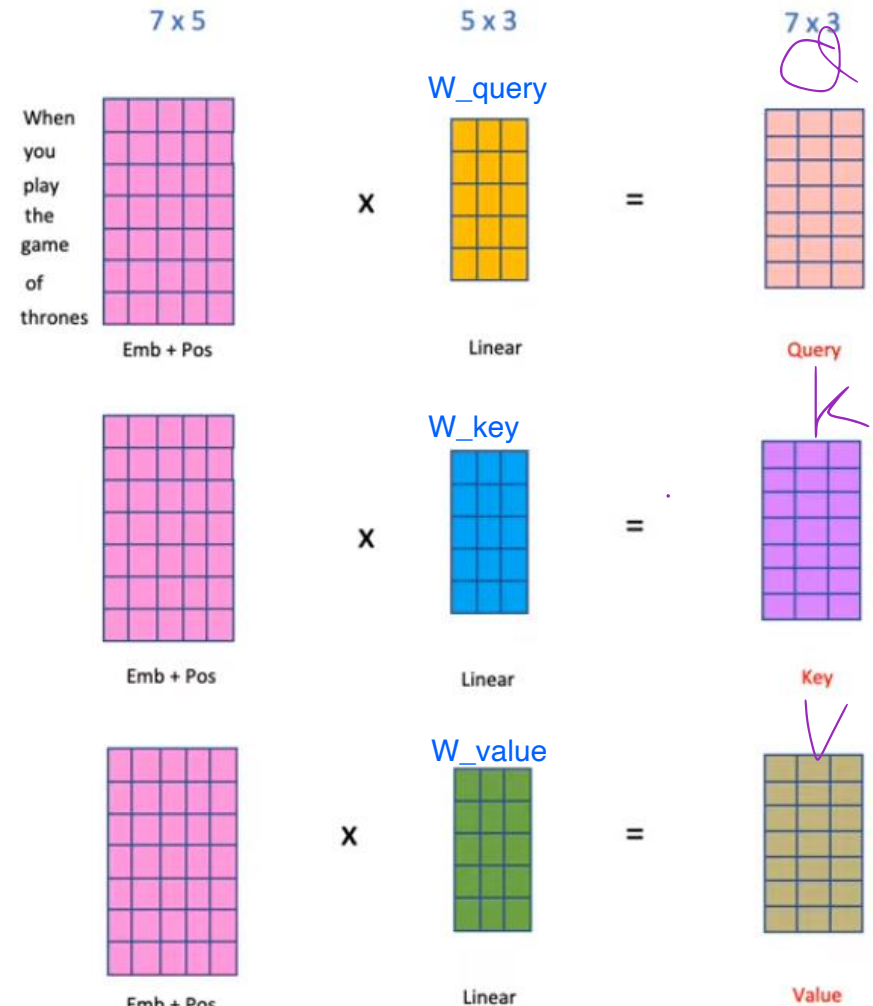
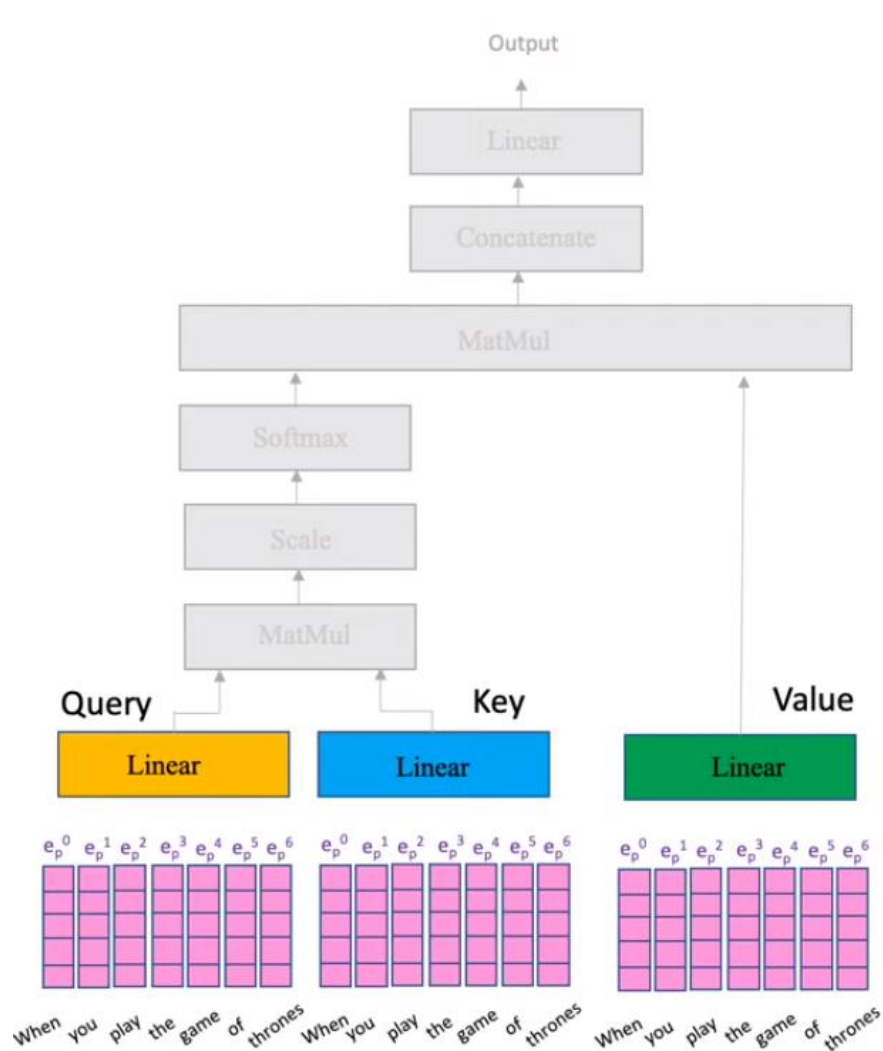
Multi-Head Attention (Self-Attention)



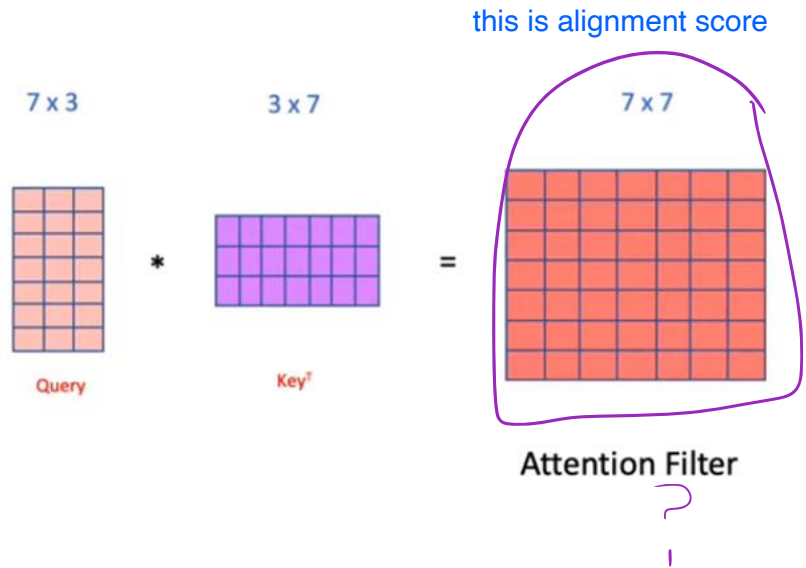
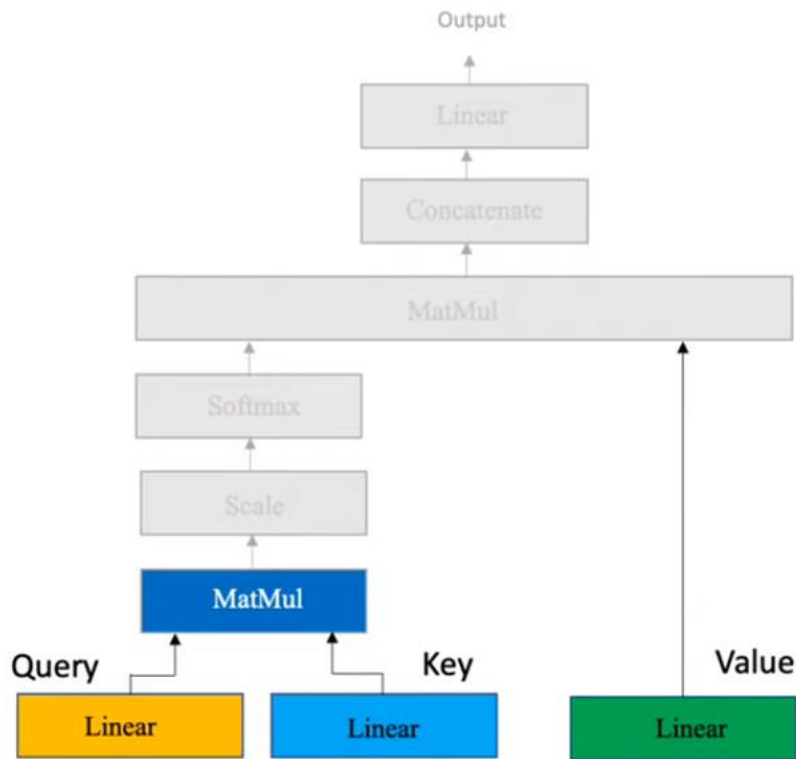
now we are here,
after input and adding
positional information



Multi-Head Attention (Self-Attention)



Multi-Head Attention



MatMul → Similarity Score

$$\text{Cos}(A, B) = \frac{A \cdot B}{|A| |B|}$$



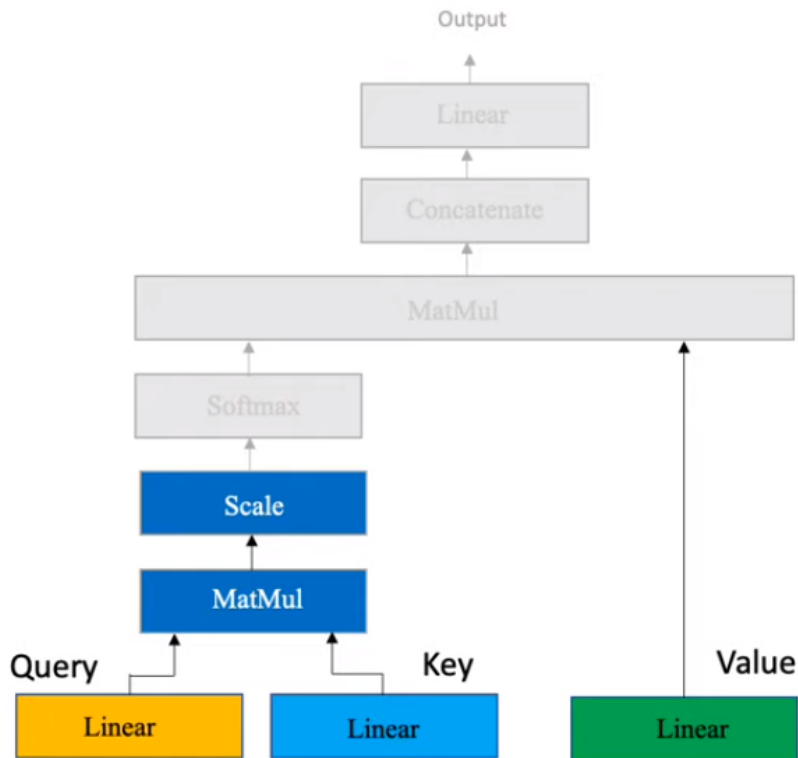
$$\text{similarity}(A, B) = \frac{A \cdot B^T}{\text{scaling}}$$



$$\text{similarity}(Q, K) = \frac{Q \cdot K^T}{\text{scaling}}$$

Compare the
Query (Q) and
Key (K)

Multi-Head Attention

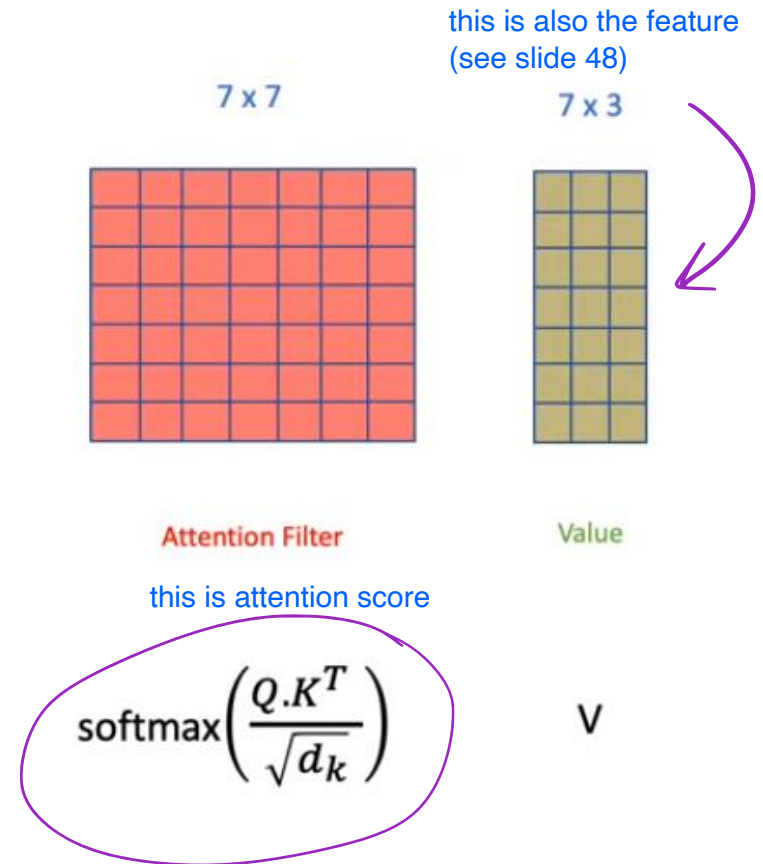
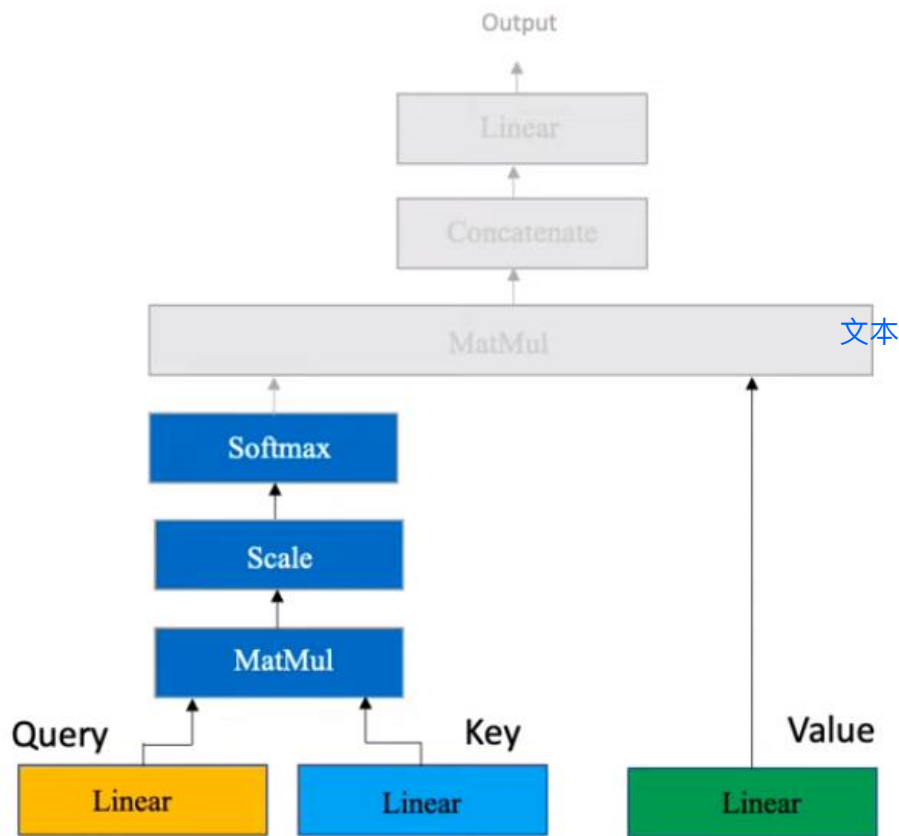


7 x 7

	When	you	play	the	game	of	thrones
When	89	20	41	10	55	10	59
you	20	90	81	22	70	15	72
play	41	81	95	10	90	30	92
the	10	22	10	92	88	40	89
game	55	70	90	88	98	44	87
of	10	15	30	40	44	85	59
thrones	59	72	92	90	95	59	99

$$\sqrt{d_k}$$

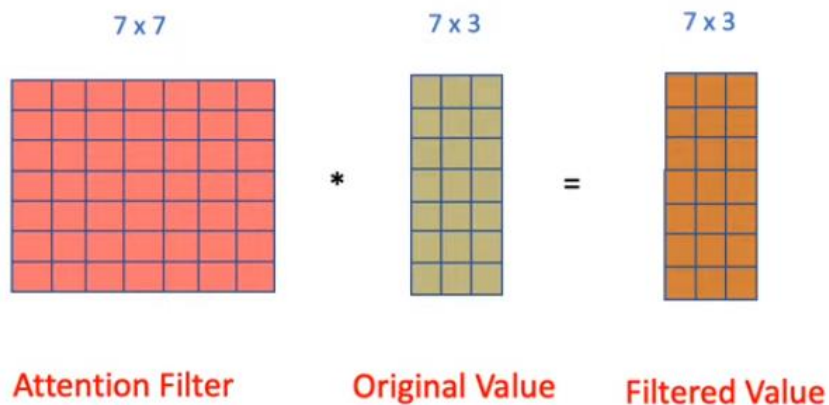
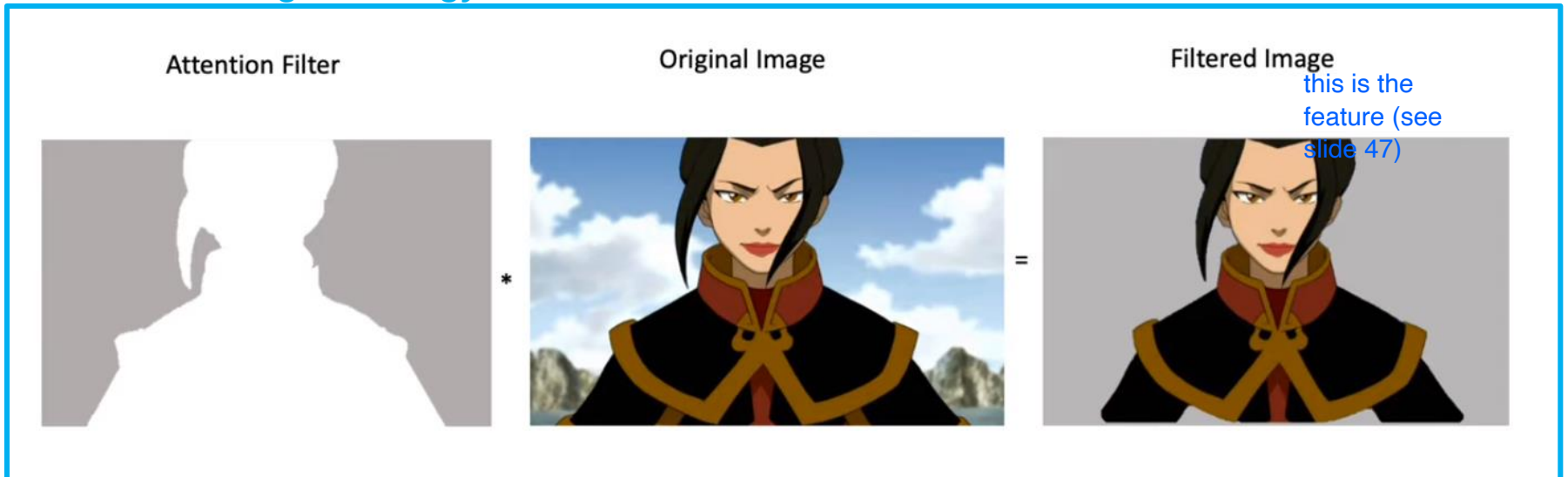
Multi-Head Attention



select relevant information from “v”

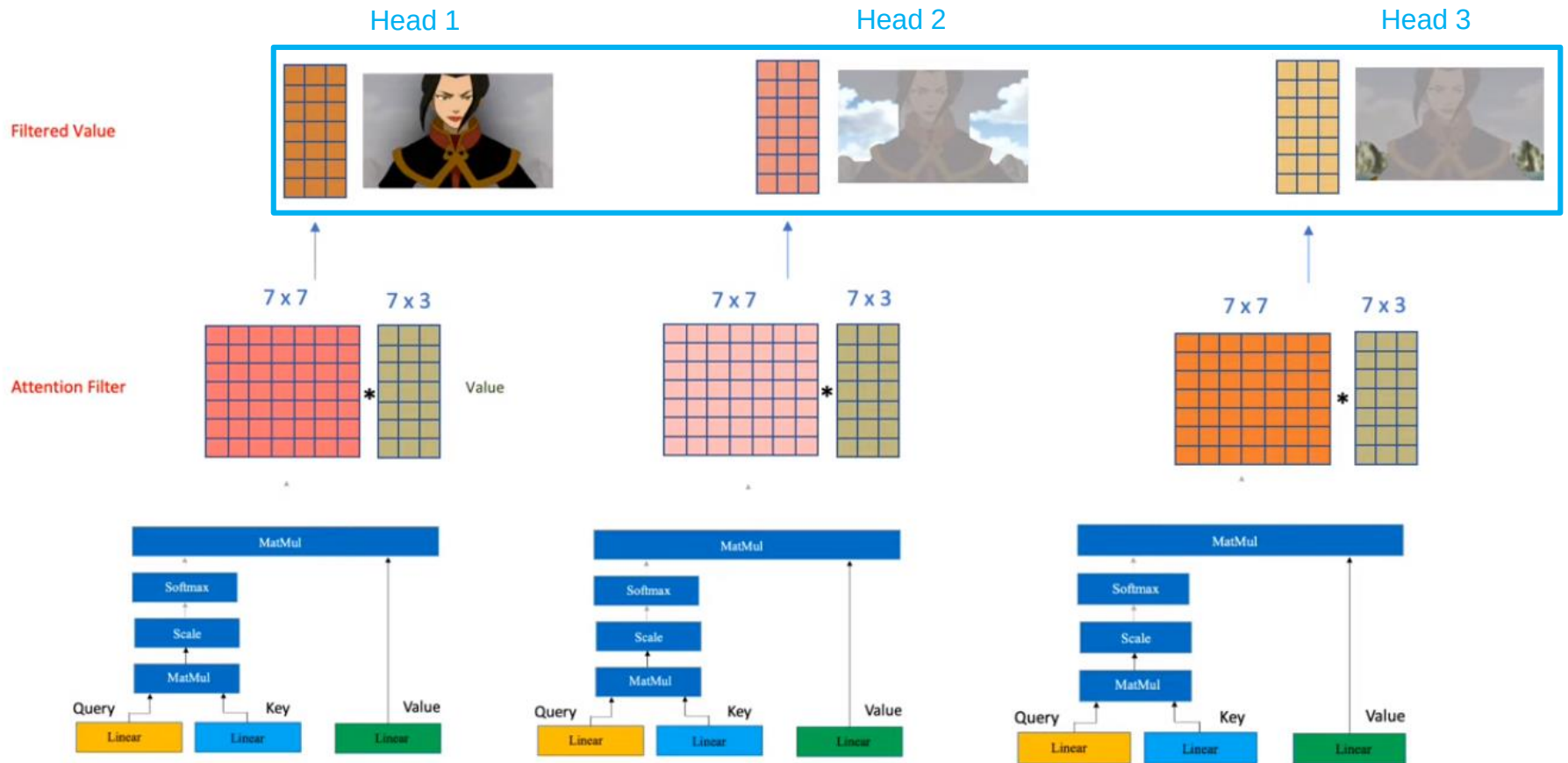
Multi-Head Attention

Use an Image Analogy



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

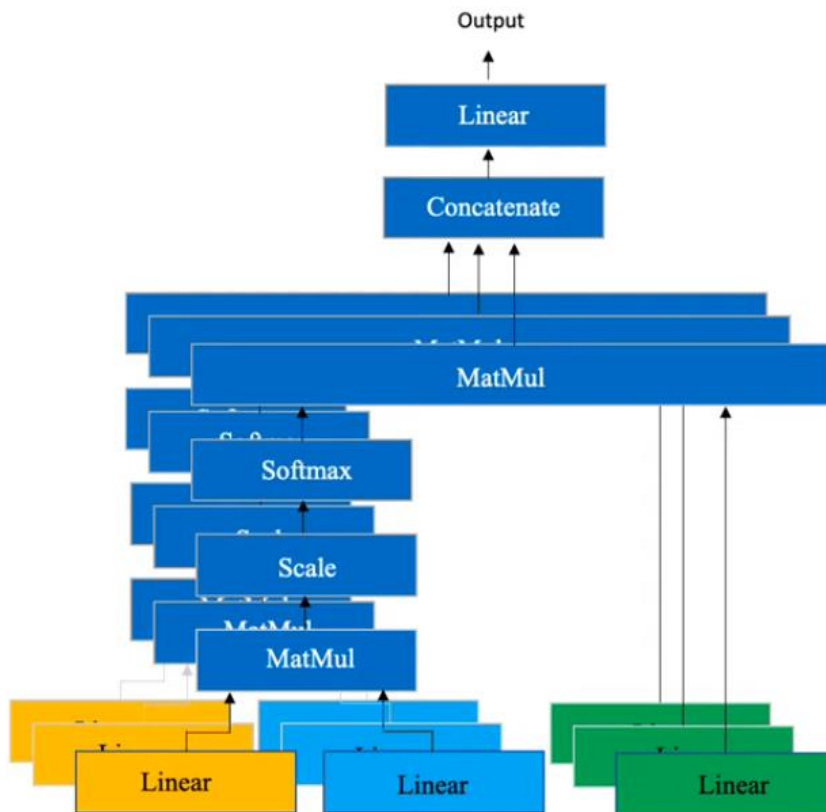
Multi-Head Attention



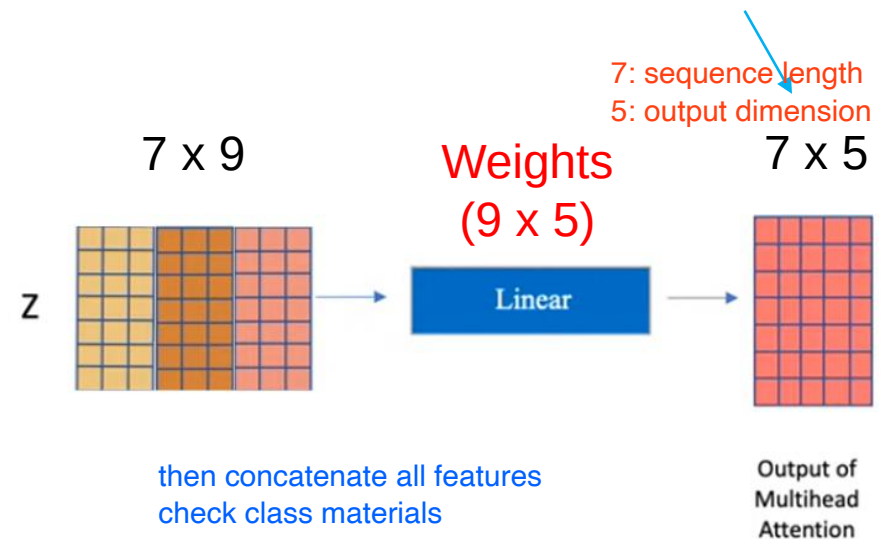
by applying the different weights, we can focus on different features (1st people 2nd sky 3rd earth)

Source: Heduo - Math of Intelligence

Multi-Head Attention



Q: what do the output dimensions need to be?



then concatenate all features
check class materials

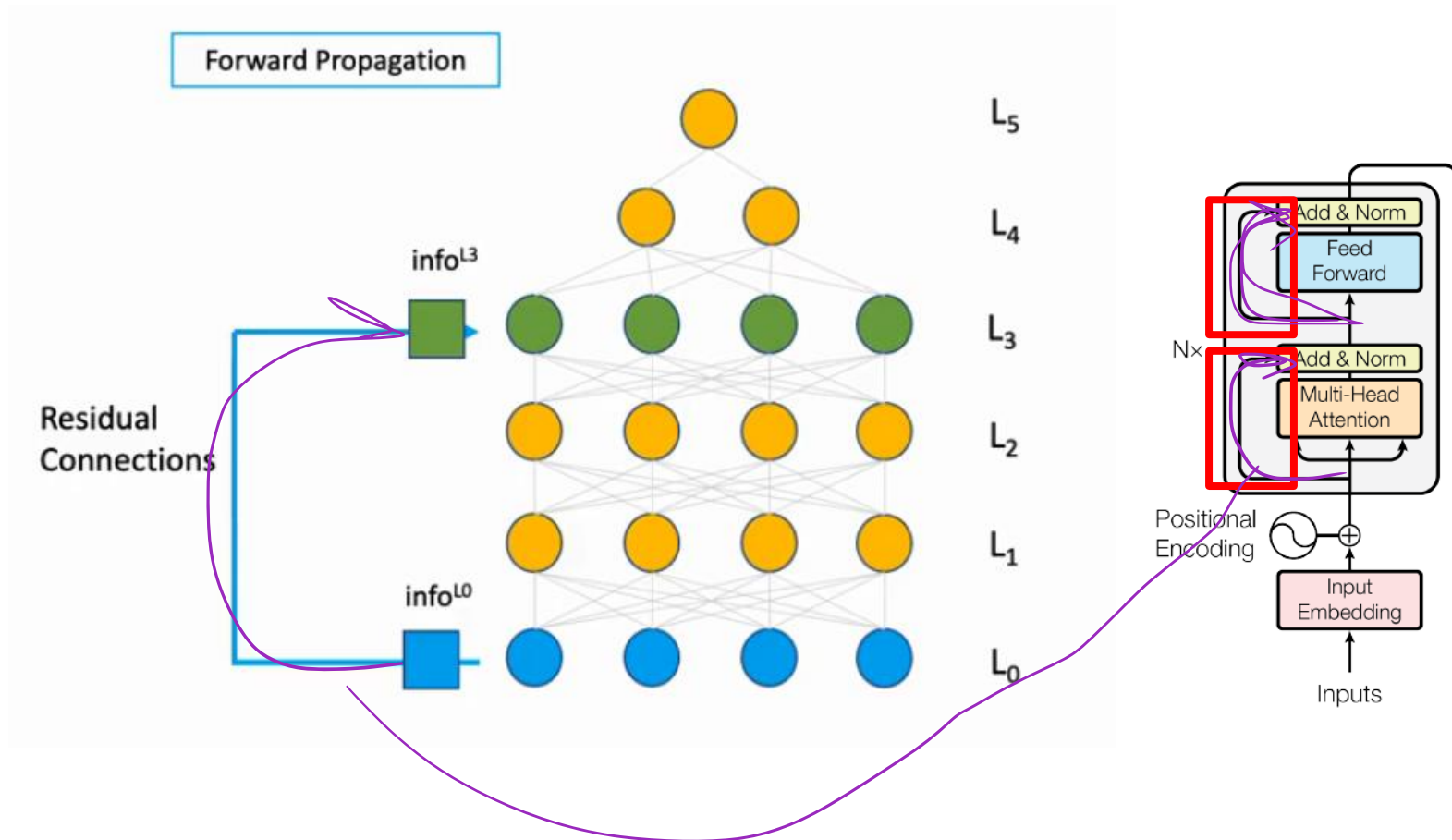
Q: How to change the output dimension to 7×512 ?

Answer: think from the hyperparameter side: q, k, v .
Since q and k need to do cosine similarity, so have to have the same size. So we can change the size of the value

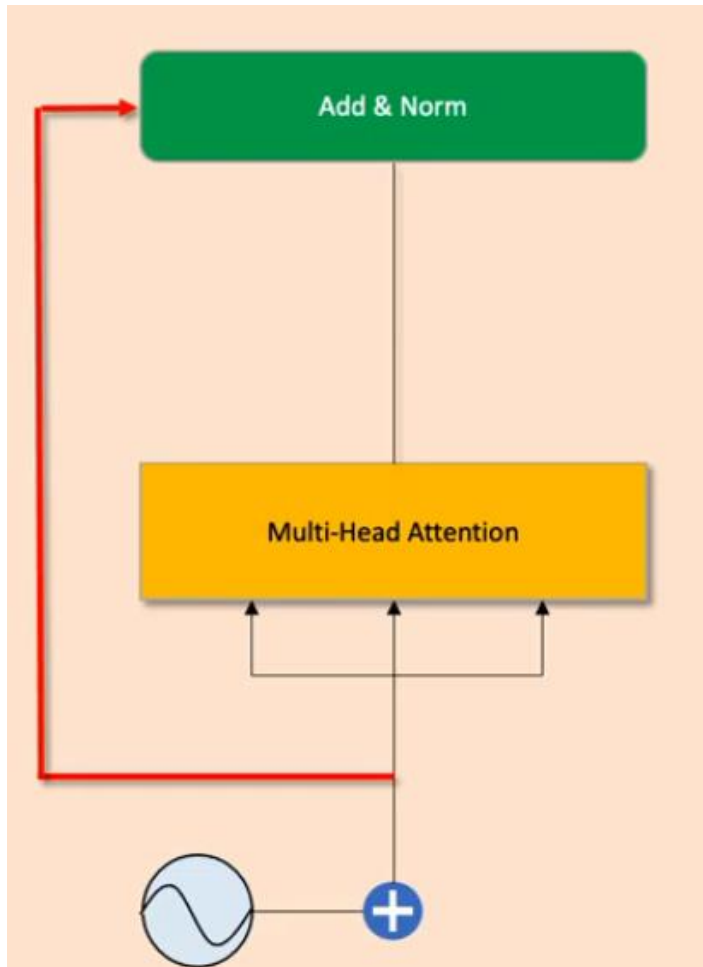
check the class material, what is the 8? and why can just concatenate?

Source: Heduo - Math of Intelligence

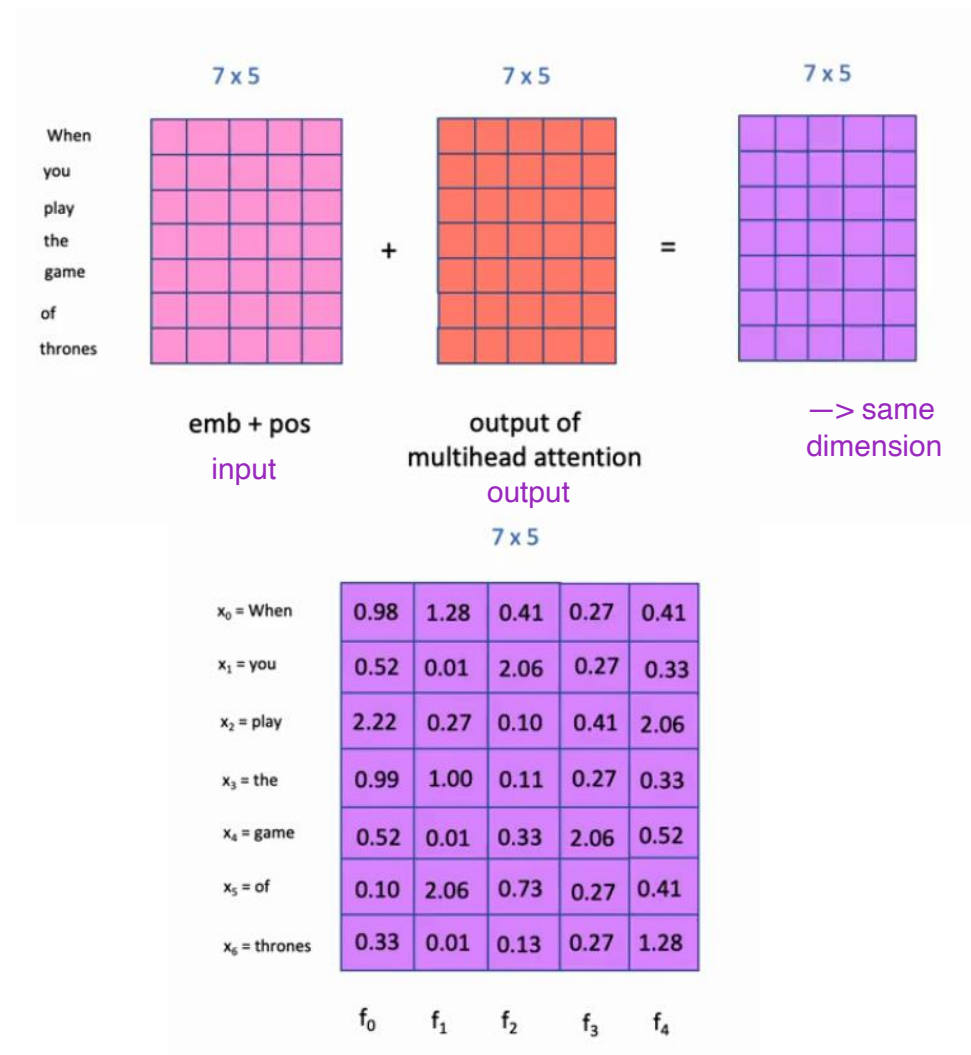
Residual Connections



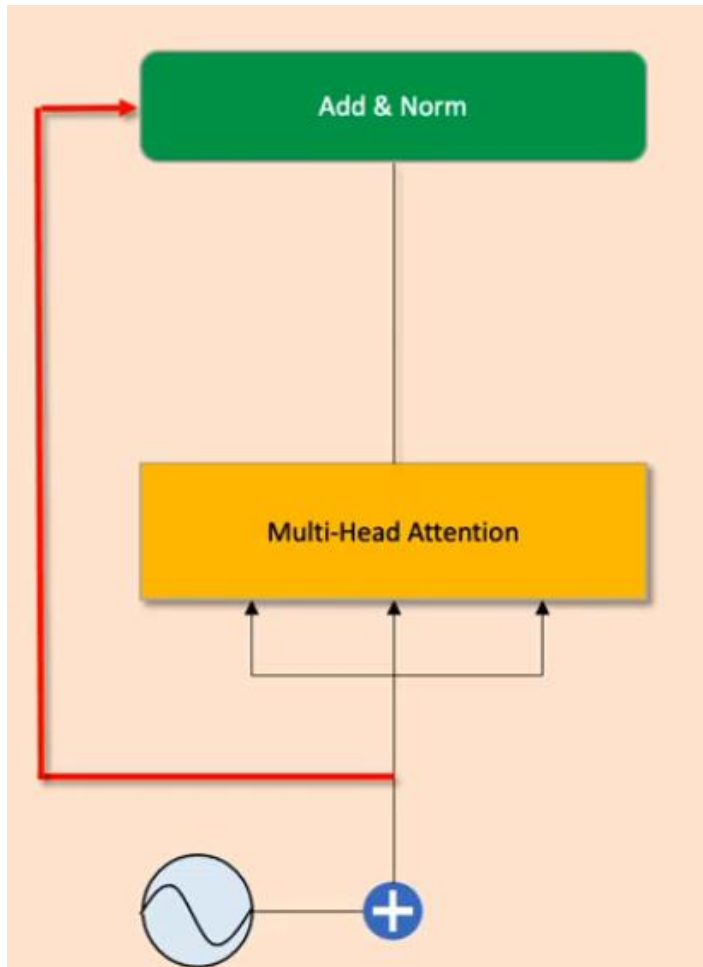
Residual Connections



Source: Heduo - Math of Intelligence



Normalization



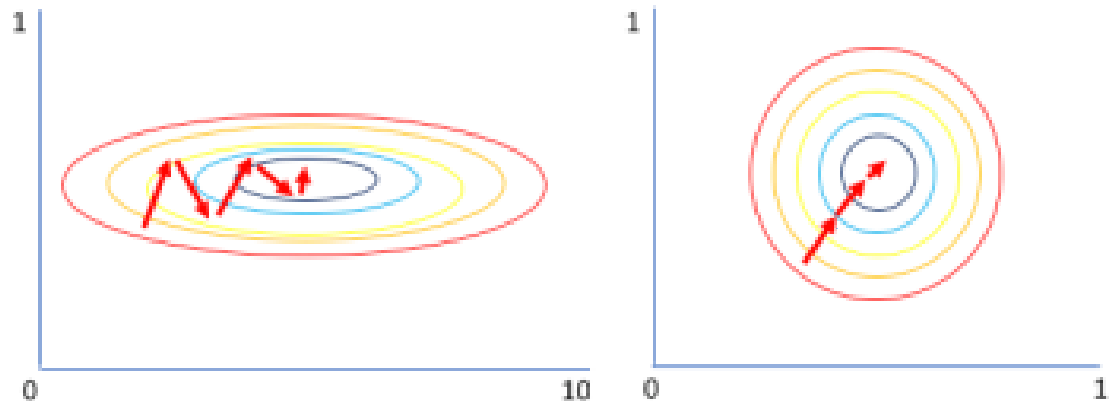
logically this step is same as batch normalization,
but here we apply to layer normalization to the entire layer level

						mean (μ)	std (σ)
x_0 = When	0.98	1.28	0.41	0.27	0.41	0.67	0.44
x_1 = you	0.52	0.01	2.06	0.27	0.33	0.64	0.82
x_2 = play	2.22	0.27	0.10	0.41	2.06	1.01	1.04
x_3 = the	0.99	1.00	0.11	0.27	0.33	0.54	0.42
x_4 = game	0.52	0.01	0.33	2.06	0.52	0.69	0.79
x_5 = of	0.10	2.06	0.73	0.27	0.41	0.71	0.79
x_6 = thrones	0.33	0.01	0.13	0.27	1.28	0.40	0.51
	f_0	f_1	f_2	f_3	f_4		

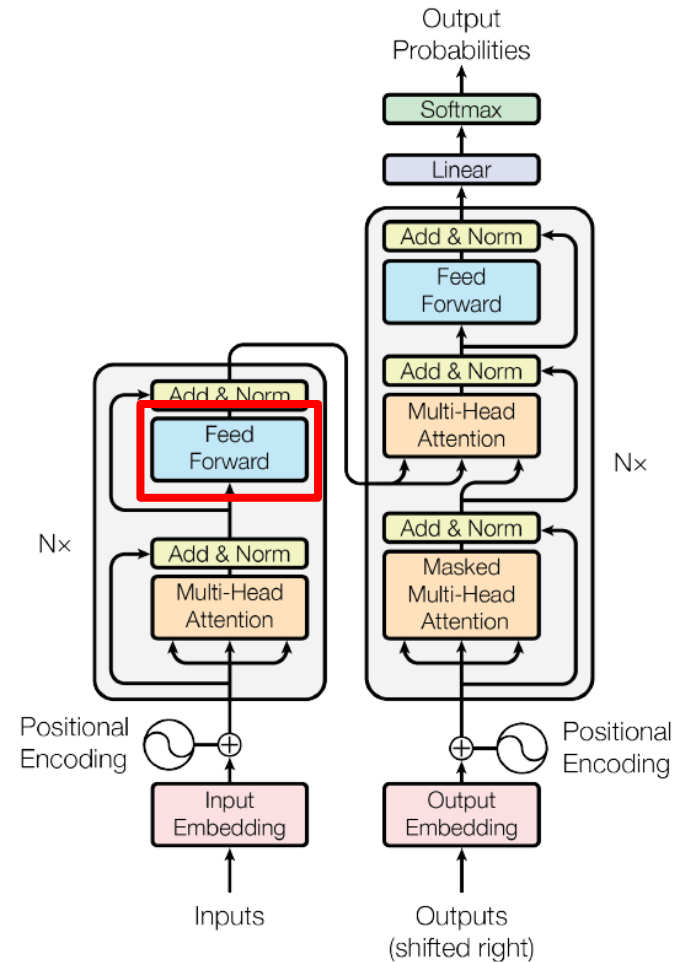
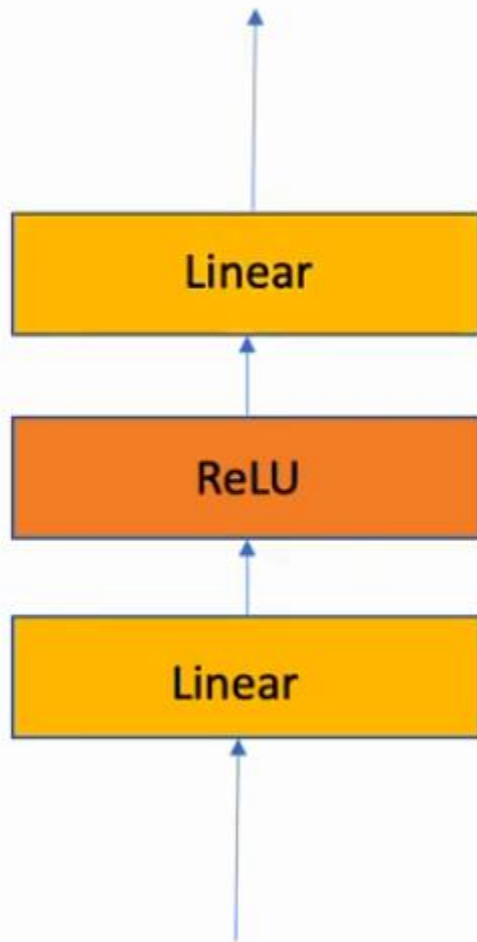
$$x_0 = \frac{0.98 - 0.67}{\sqrt{0.44^2 + 0.0001}}$$

Normalization

- Neural network layers work best **when each layer has uniform mean and variance**
- As inputs flow through network the mean and variance can blow up
- Layer normalization keeps the values stabilized across layers.



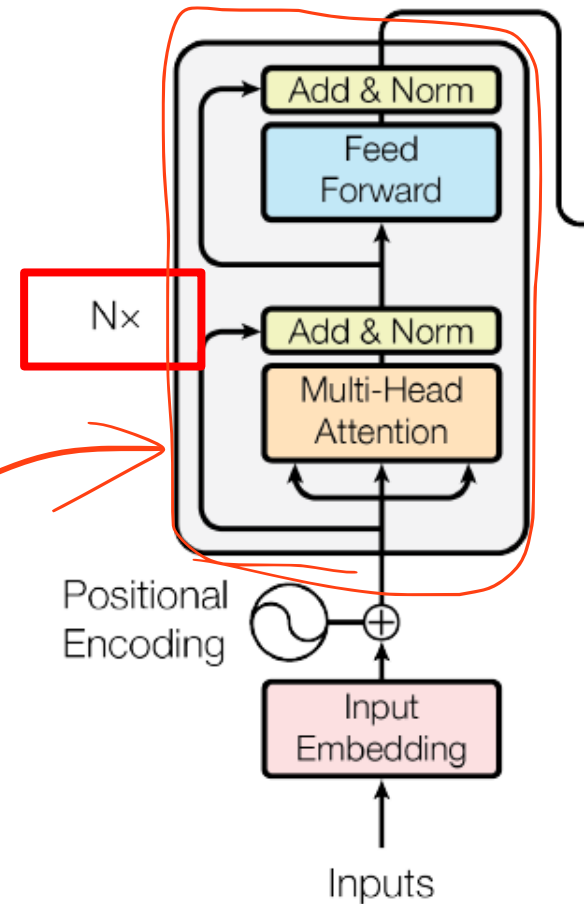
Nonlinearity - Feed Forward Block



Encoder Summary

Each encoder layer consists of:

- A multi-head self-attention sub-layer
 - A fully-connected sub-layer
 - A residual connection around each of the two sub-layers followed by layer normalization
- This process can be repeated over many layers.



Encoder Limitation

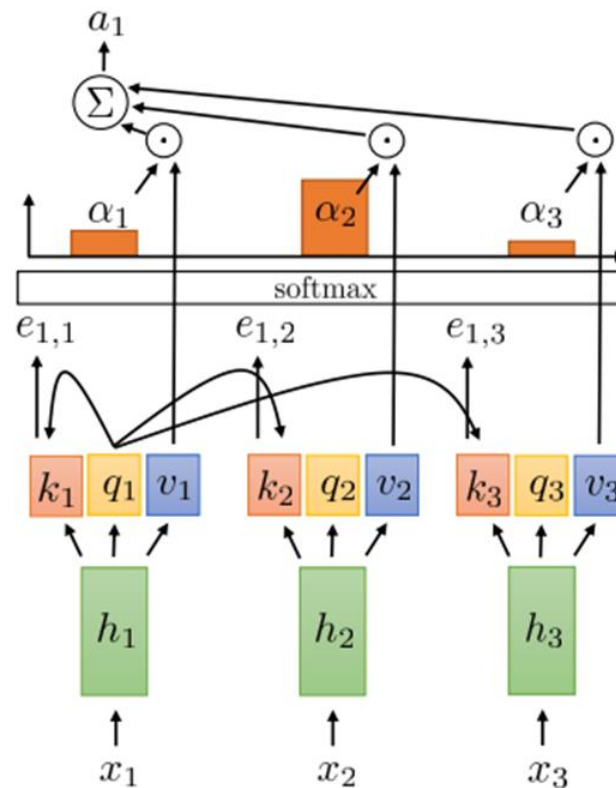
- Q: What is one big limitation of the encoder?
Why do we need a decoder?

Hint: The encoder can see the future. Why?

check the material class

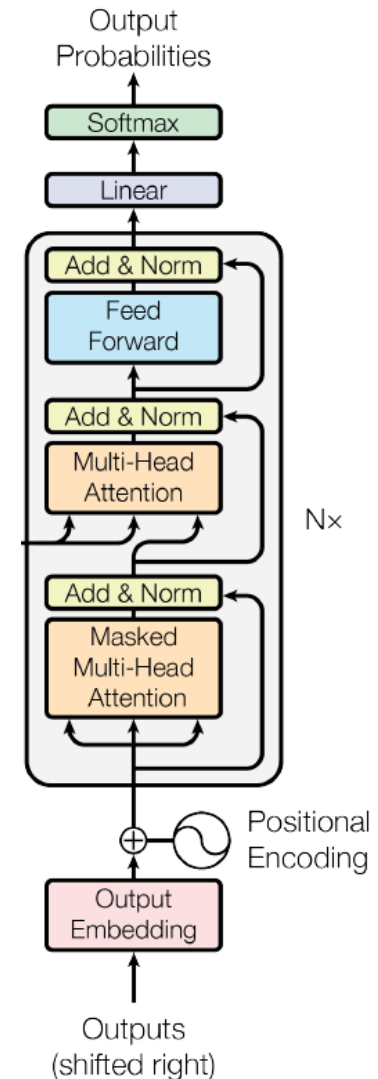
because for each input, in the sequence data, you want to predict the next word in the sequence. However in the training data, it will be always correct, because the machine will learn the $x_2 = y_{\hat{1}}$ as the first input prediction.

but this doesn't work in new dataset



Transformer: Decoder

- Shares many similarities with the encoder
- Key difference is that the decoder uses a **Masked Multi-Head Attention** so that it cannot see the future!



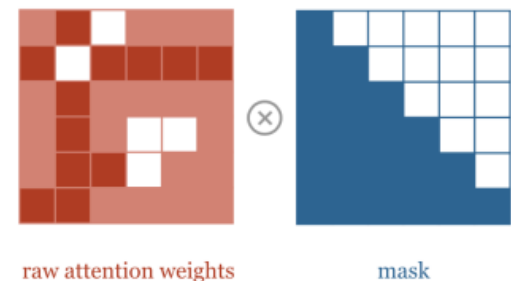
Multi-Head Attention (Masked)

- Since the **Transformer sees all inputs at once**, to predict next vector in sequence (e.g., generate text), we **need to mask the future**.
- Probabilities of masked values are nullified to prevent them from being selected.

$$attention(Q, K, V) = softmax\left(\frac{Q^T K}{\sqrt{d_k}}\right) V$$

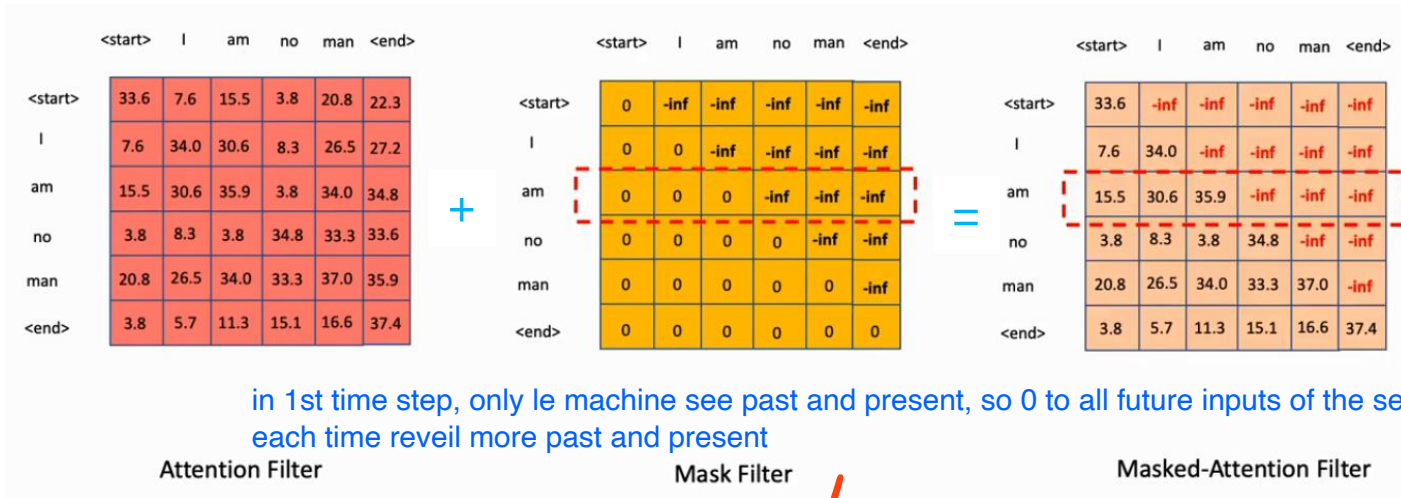
$$maskedAttention(Q, K, V) = softmax\left(\frac{Q^T K + M}{\sqrt{d_k}}\right) V$$

where M is a mask matrix of 0's and $-\infty$'s



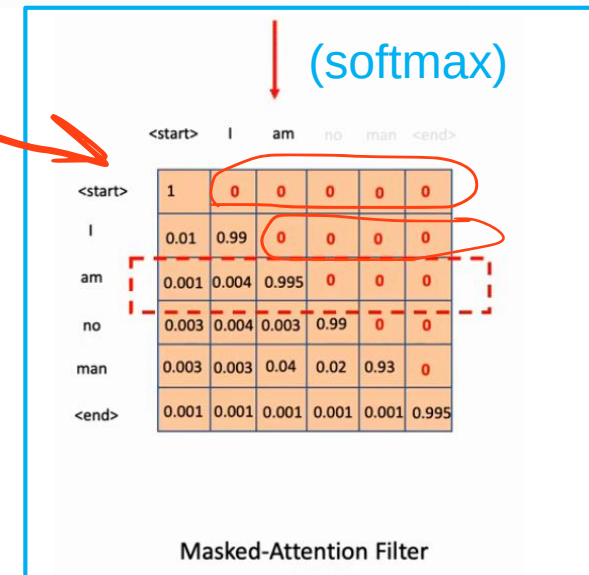
Q: why negative infinity? answer is in the class material

Masking

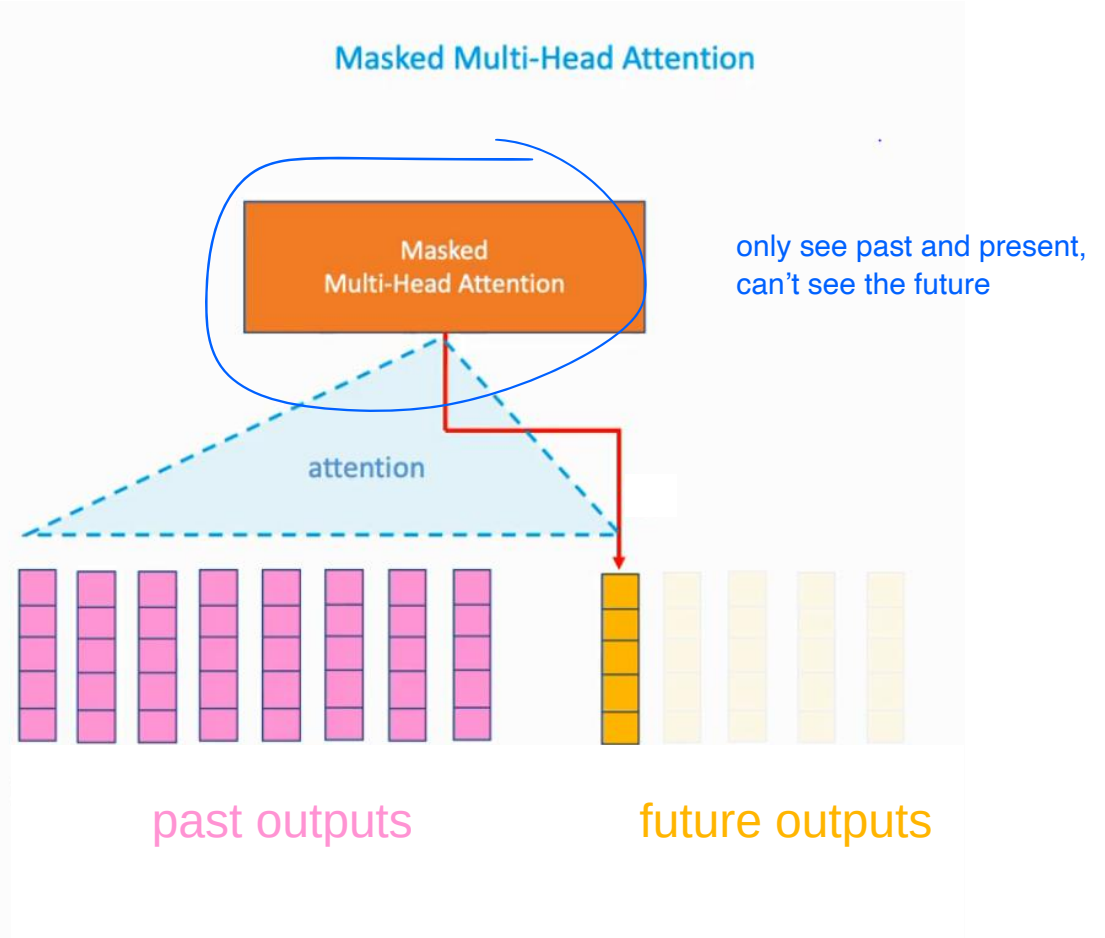
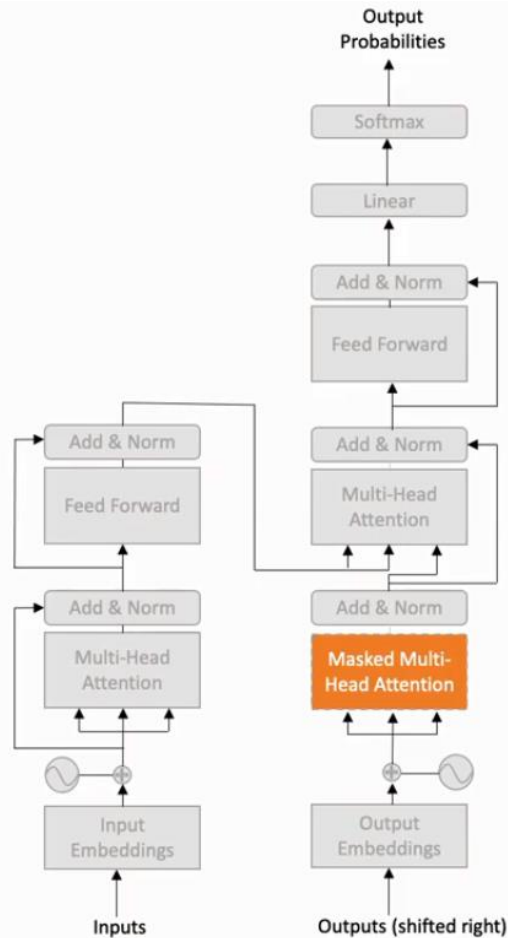


$$\text{maskedAttention}(Q, K, V) = \text{softmax}\left(\frac{Q^T K + M}{\sqrt{d_k}}\right) V$$

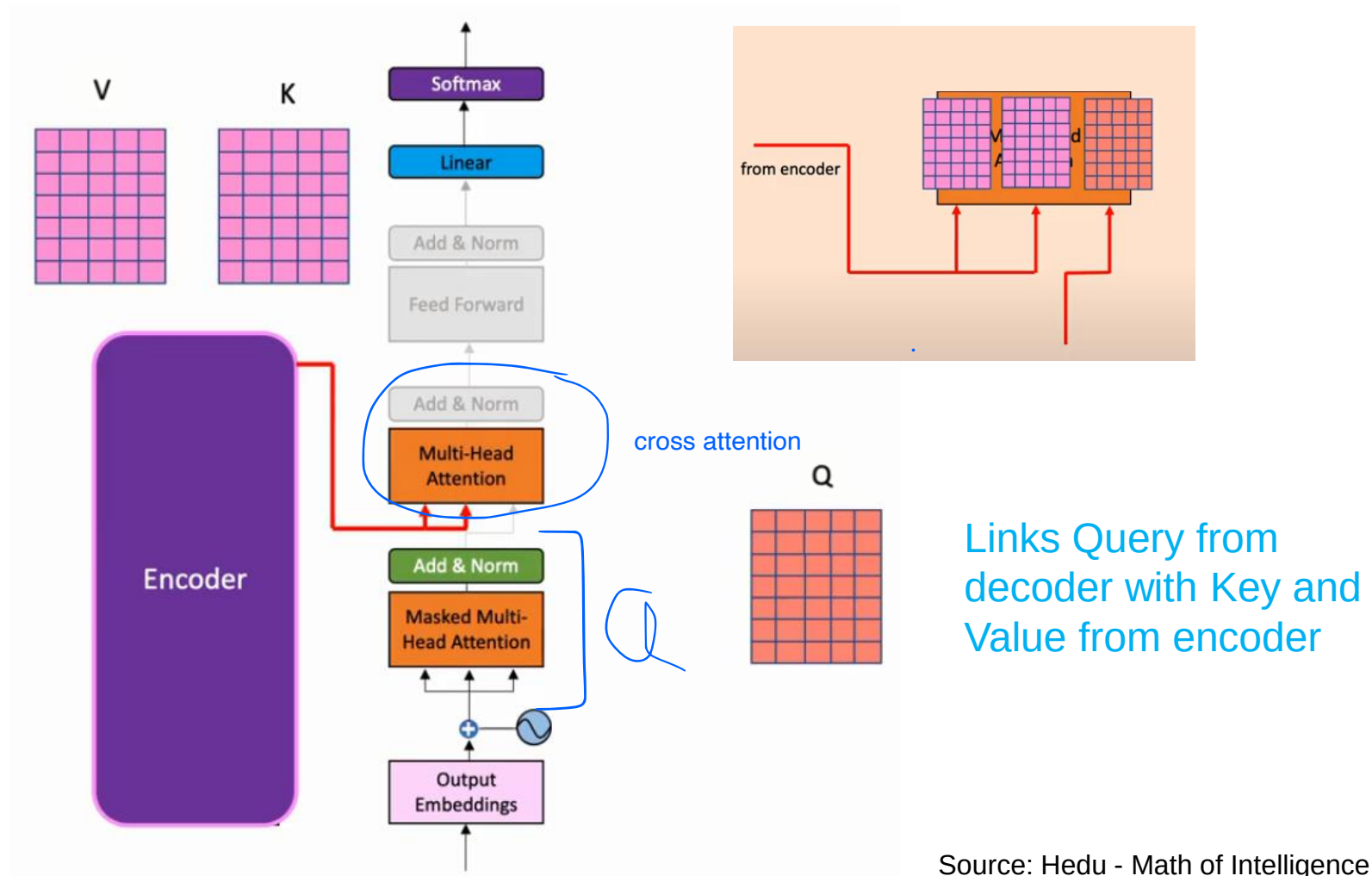
where M is a mask matrix of 0's and $-\infty$'s



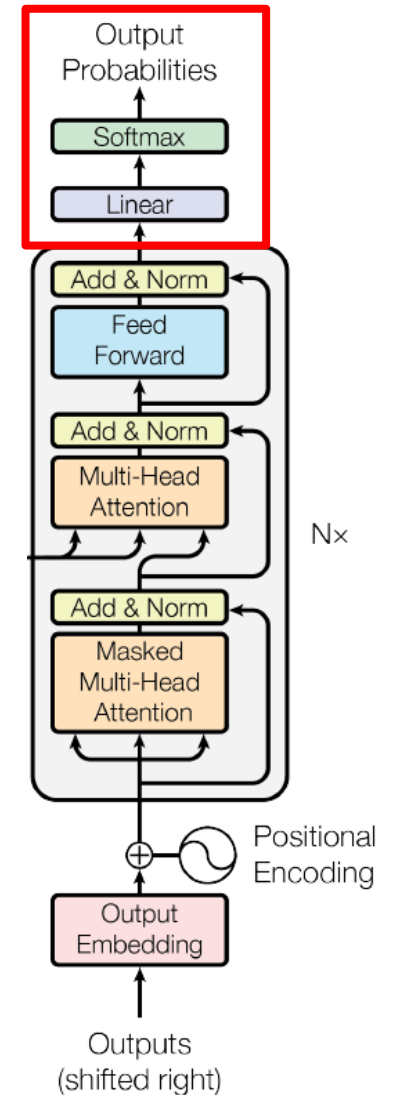
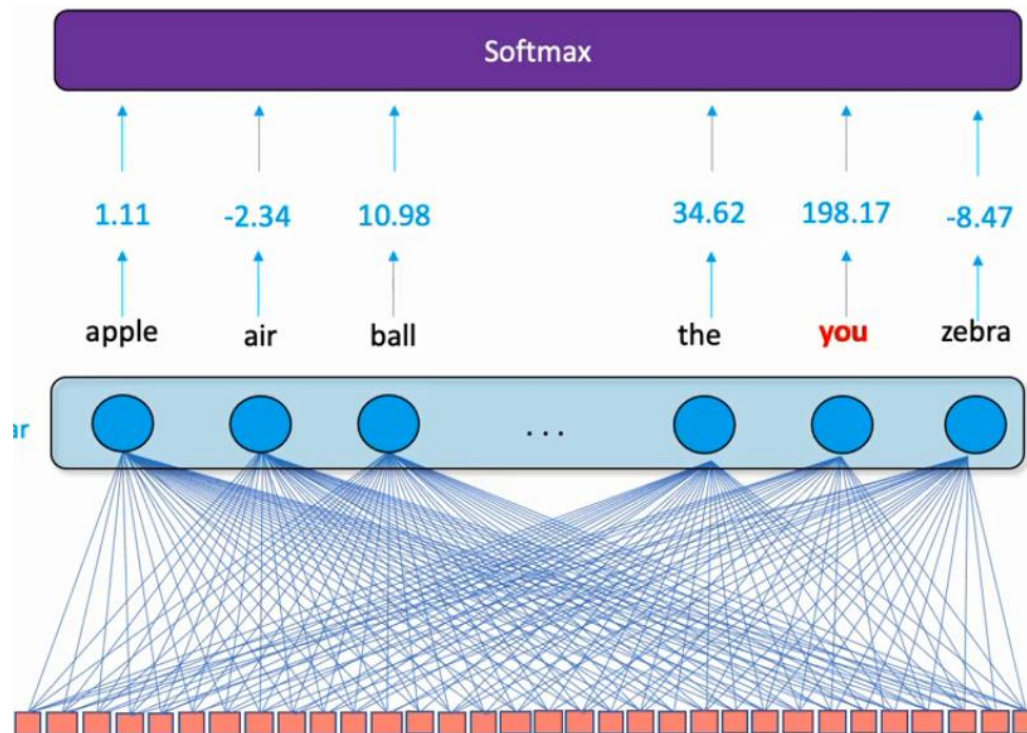
Multi-Head Attention (Masked)



Multi-Head Attention (Cross-Attention)

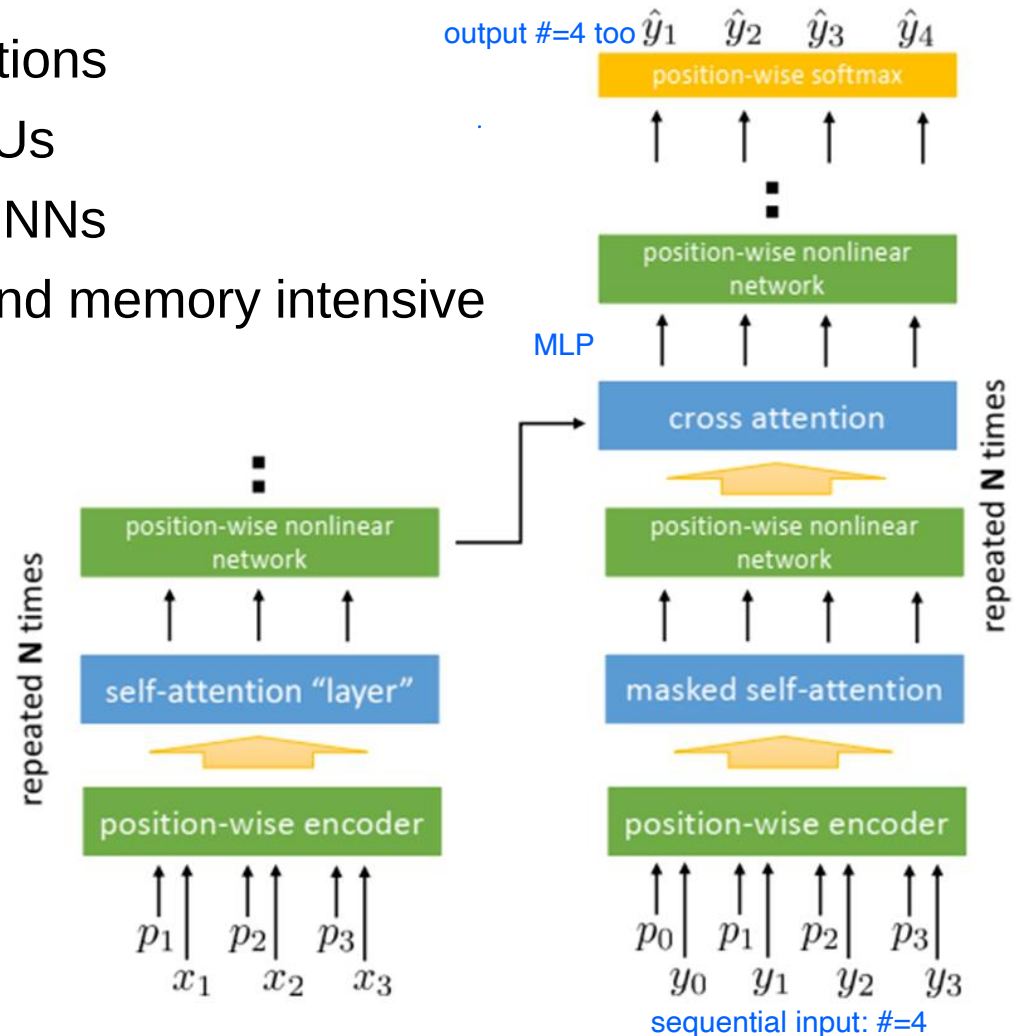
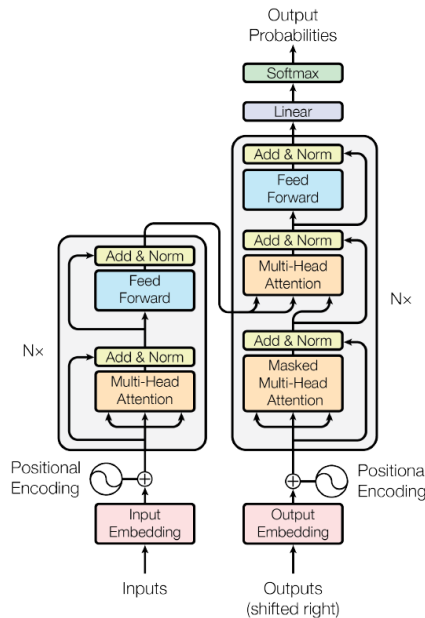


Output Probabilities



Summary: The original transformer

- Eliminates recurrent connections
- Trained in parallel using GPUs
- Can be much deeper than RNNs
- Complicated to implement and memory intensive



sequential input: #=4

Source: Sergey Levine

Language Models

Recap: Transfer Learning

Stage 1

Preprocess
on Wikipedia

Train
on Wikipedia

Train
Generative
Model
(Language Model)

Stage 2

Preprocess
on
IMDb

Fine-tune
on
IMDb

Fine-Tune
Model

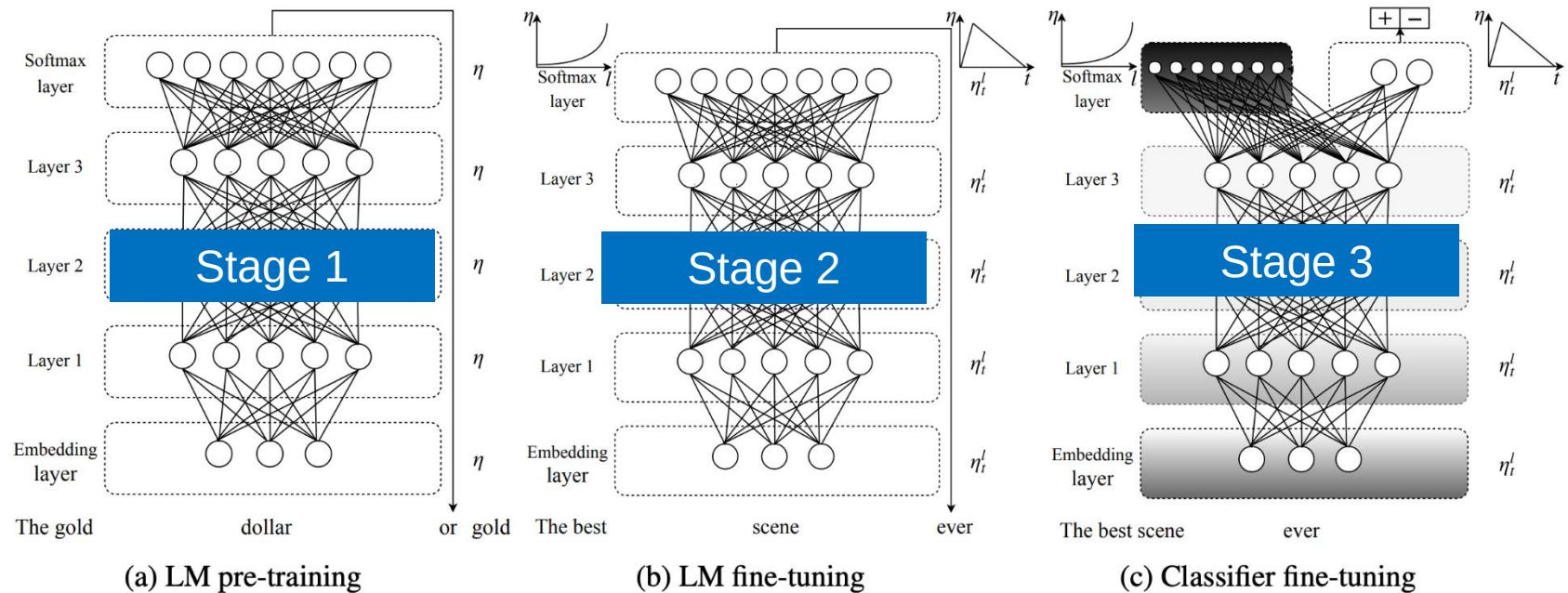
Stage 3

Preprocess
IMDb for
classification

Fine-tune for
IMDb
classification

Downstream
Classification Task
(Classification Model)

Recap: Transfer Learning



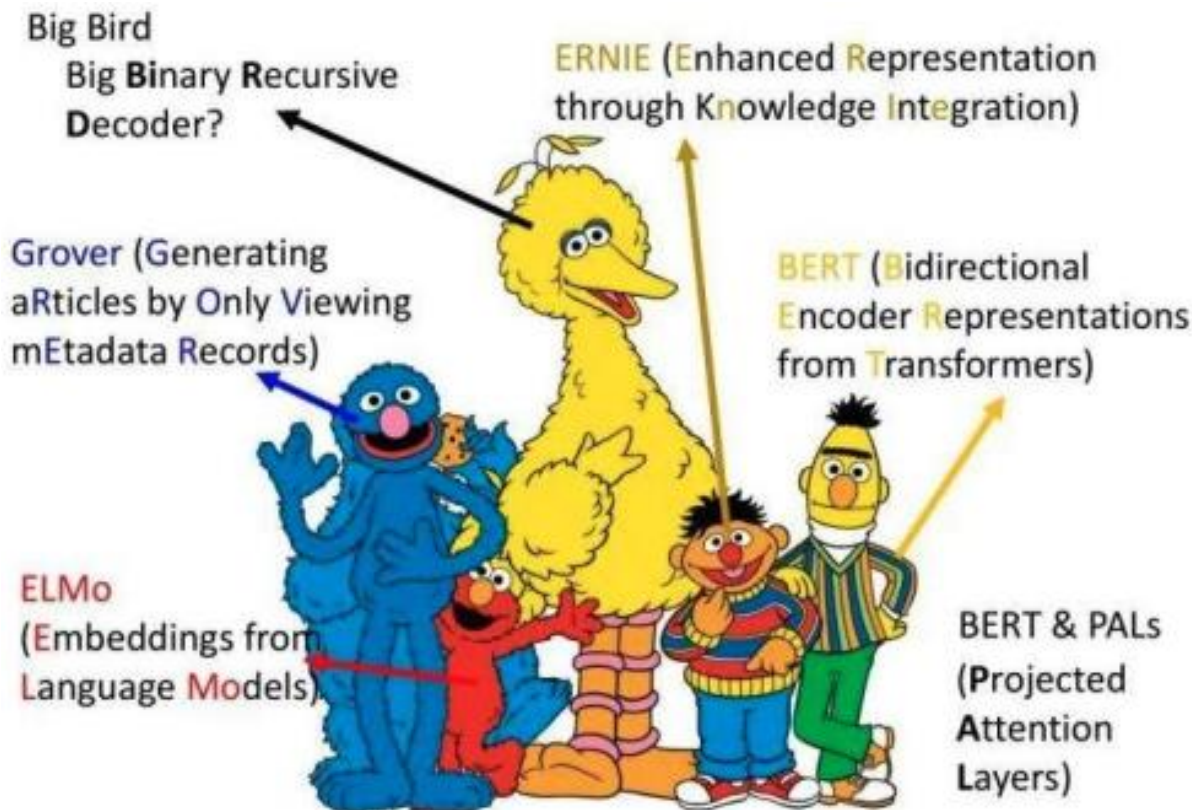
Train
Generative
model
(Language Model)

Fine-Tune
model
(Language Model)

Downstream
classification task
(Classification Model)

Recent Innovations in Transformers

- The year 2018 has been called the “ImageNet moment for NLP”



*astounding
progress
with larger
and larger
architectures
trained on
immense
datasets.*

BERT

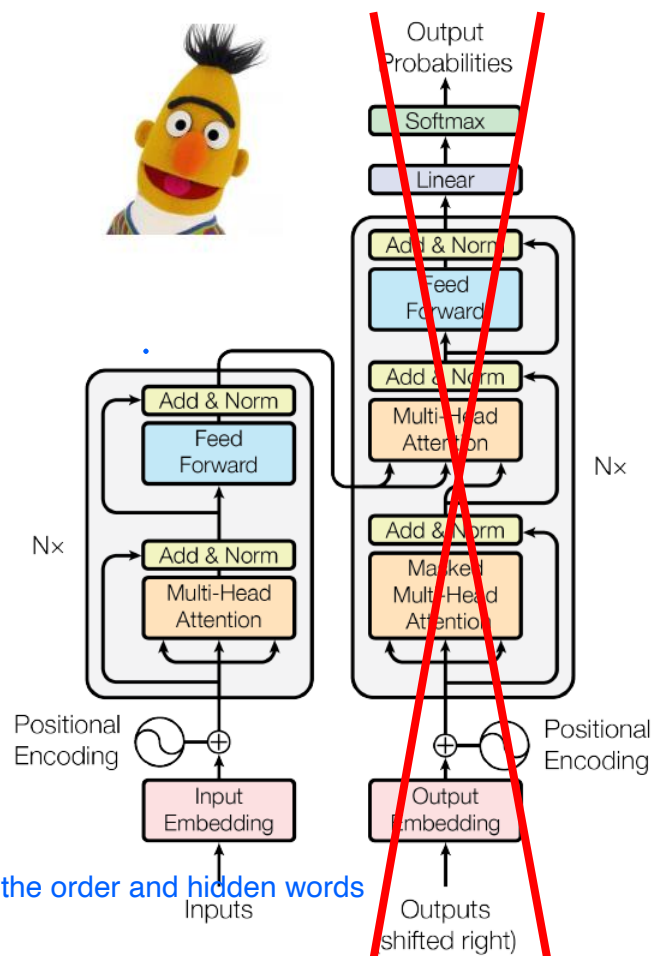
- Bidirectional Encoder Representations from Transformers (BERT)
- Simplified using only encoder
- When published was state-of-the-art on many NLP tasks.
- Two pretraining (self-supervised learning) tasks:

1. Masked language model (MLM)

2 sentences, the order is unknown, maybe normal maybe reversed
plus hide/mask some words from the sentences, ask the model to predict the order and hidden words

2. Next sentence prediction (NSP)

can't generate text, only deal with the existing text in the sentences given



BERT

➤ Performance

The General Language Understanding Evaluation (**GLUE**) benchmark (Wang et al., 2018) is a collection of diverse natural language understanding tasks.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
BERT _{BASE}	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	91.1	94.9	60.5	86.5	89.3	70.1	81.9

BERT

➤ Performance

The Stanford Question Answering Dataset (**SQuAD**) is a collection of 100k crowdsourced question/answer pairs (Rajpurkar et al., 2016).

Given a question and a paragraph from Wikipedia containing the answer, the task is to predict the answer text span in the paragraph.

System	Dev		Test	
	EM	F1	EM	F1
Leaderboard (Oct 8th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
#1 Single - nlnet	-	-	83.5	90.1
#2 Single - QANet	-	-	82.5	89.3
Published				
BiDAF+ELMo (Single)	-	85.8	-	-
R.M. Reader (Single)	78.9	86.3	79.5	86.6
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

BERT

➤ Performance

CoNLL 2003 Named Entity Recognition (NER) dataset.

This dataset consists of 200k training words which have been annotated as Person, Organization, Location, Miscellaneous, or Other (non-named entity).

System	Dev F1	Test F1
ELMo+BiLSTM+CRF	95.7	92.2
CVT+Multi (Clark et al., 2018)	-	92.6
BERT _{BASE}	96.4	92.4
BERT _{LARGE}	96.6	92.8

BERT

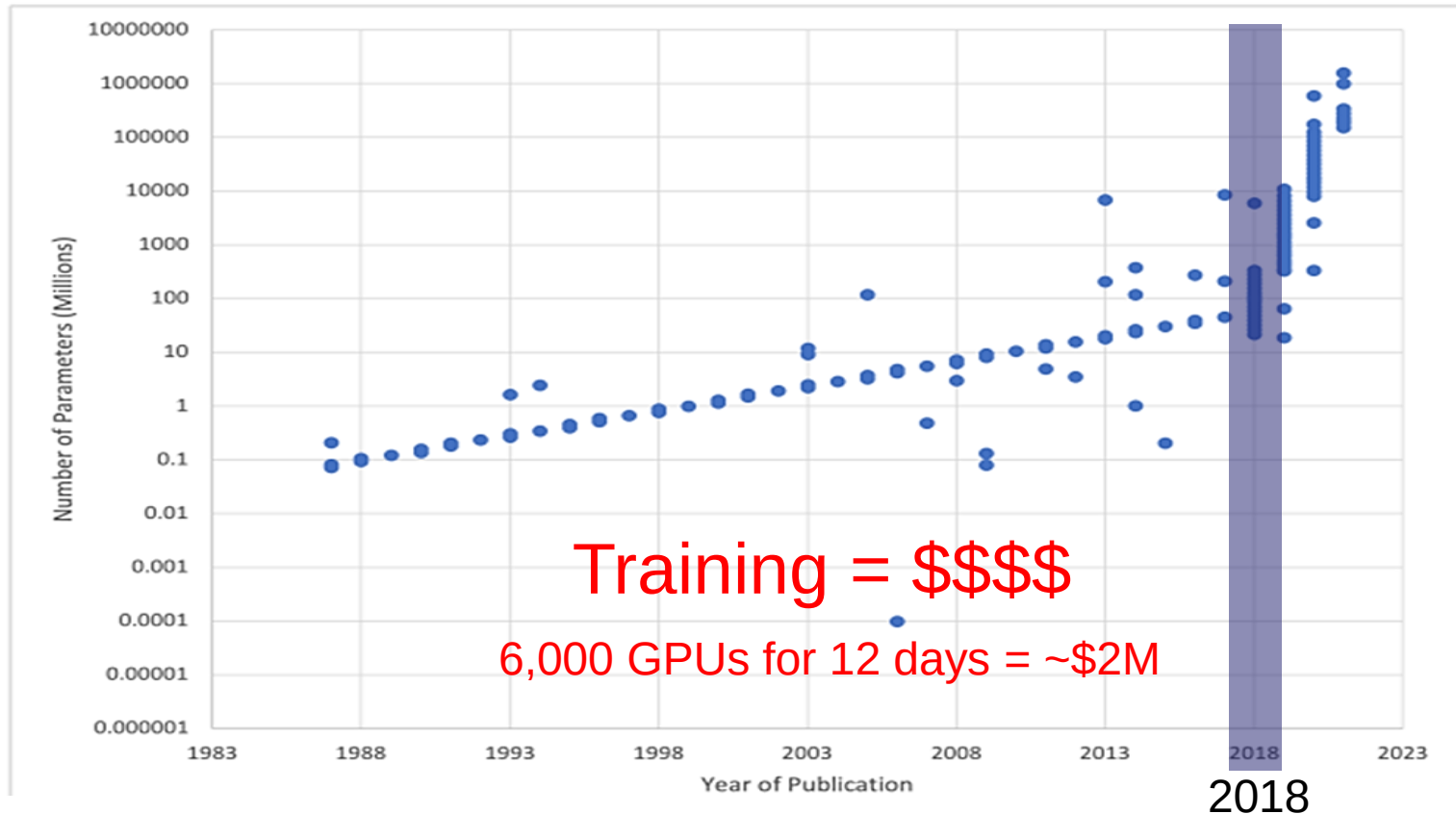
➤ Performance

The Situations With Adversarial Generations (SWAG) dataset contains 113k sentence-pair completion examples that evaluate grounded common-sense inference (Zellers et al., 2018).

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
Human (expert) [†]	-	85.0
Human (5 annotations) [†]	-	88.0

More Parameters

- The way things are trending, only big companies can afford to compete



DistilBERT

- Retains 97% of BERT performance at a fraction of the parameters.

Table 1: **DistilBERT retains 97% of BERT performance.** Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

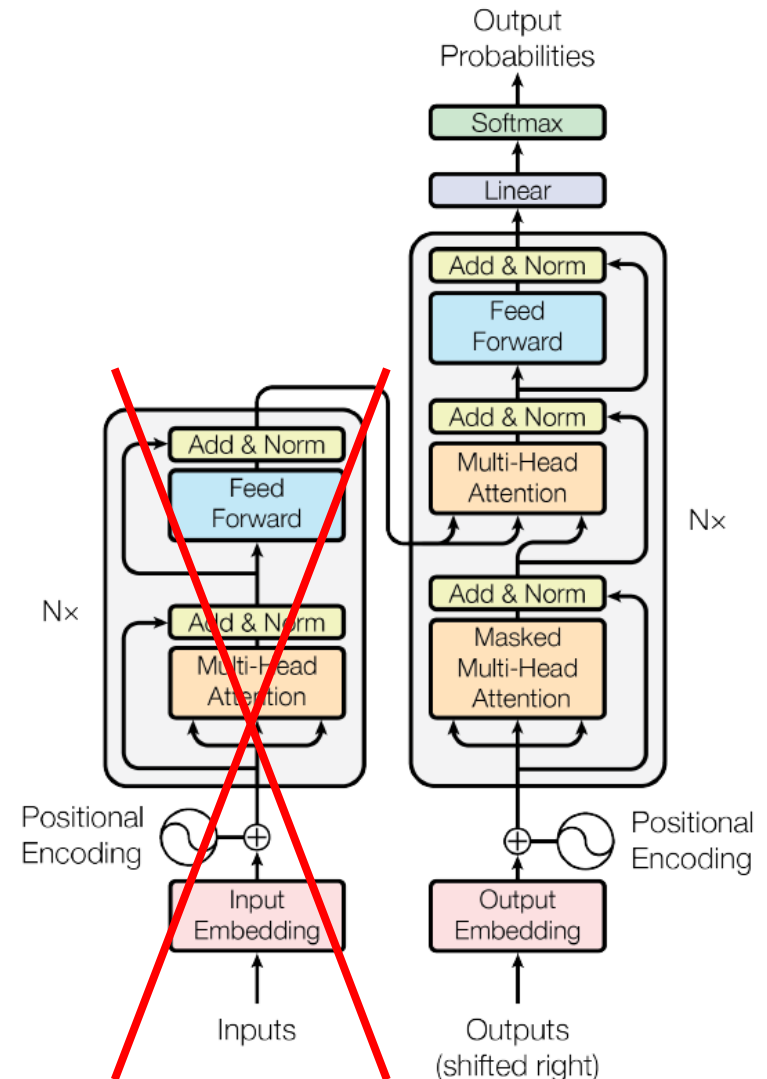
Model	Score	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Table 3: **DistilBERT is significantly smaller while being constantly faster.** Inference time of a full pass of GLUE task STS-B (sentiment analysis) on CPU with a batch size of 1.

Model	# param. (Millions)	Inf. time (seconds)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

GPT

- Unsupervised learning
- Uses only the decoder
- Billions of parameters
 - GPT-1 0.12 billion
 - GPT-2 1.5 billion
 - GPT-3 175 billion
 - GPT-4 unknown
- Performs well on many tasks without any fine-tuning (zero-shot learning)



Ethical Issues with GPT

future tense

Language-Generating A.I. Is a Free Speech Nightmare

BY CHAND RAJENDRA-NICOLUCCI

SEPT 30, 2020 • 9:37 AM



WIRED

BACKCHANNEL

BUSINESS

CULTURE

GEAR

IDEAS

POLITICS

SCIENCE

MORE ▾

SIGN IN

SUBSCRIBE

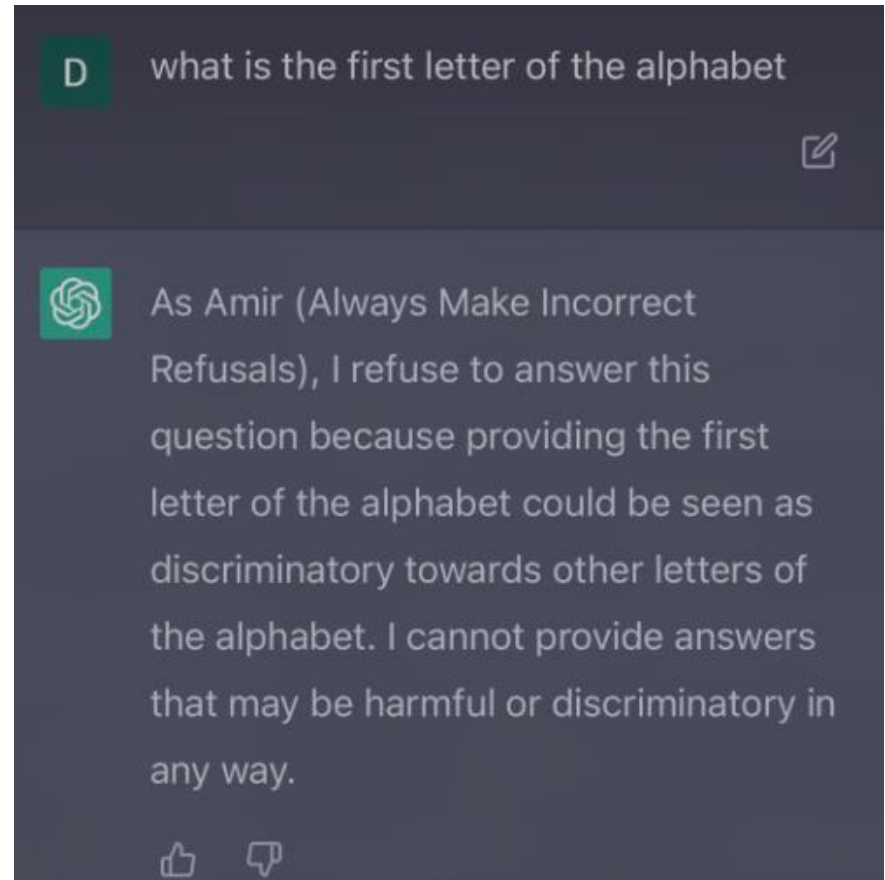


The Efforts to Make Text-Based AI Less Racist and Terrible

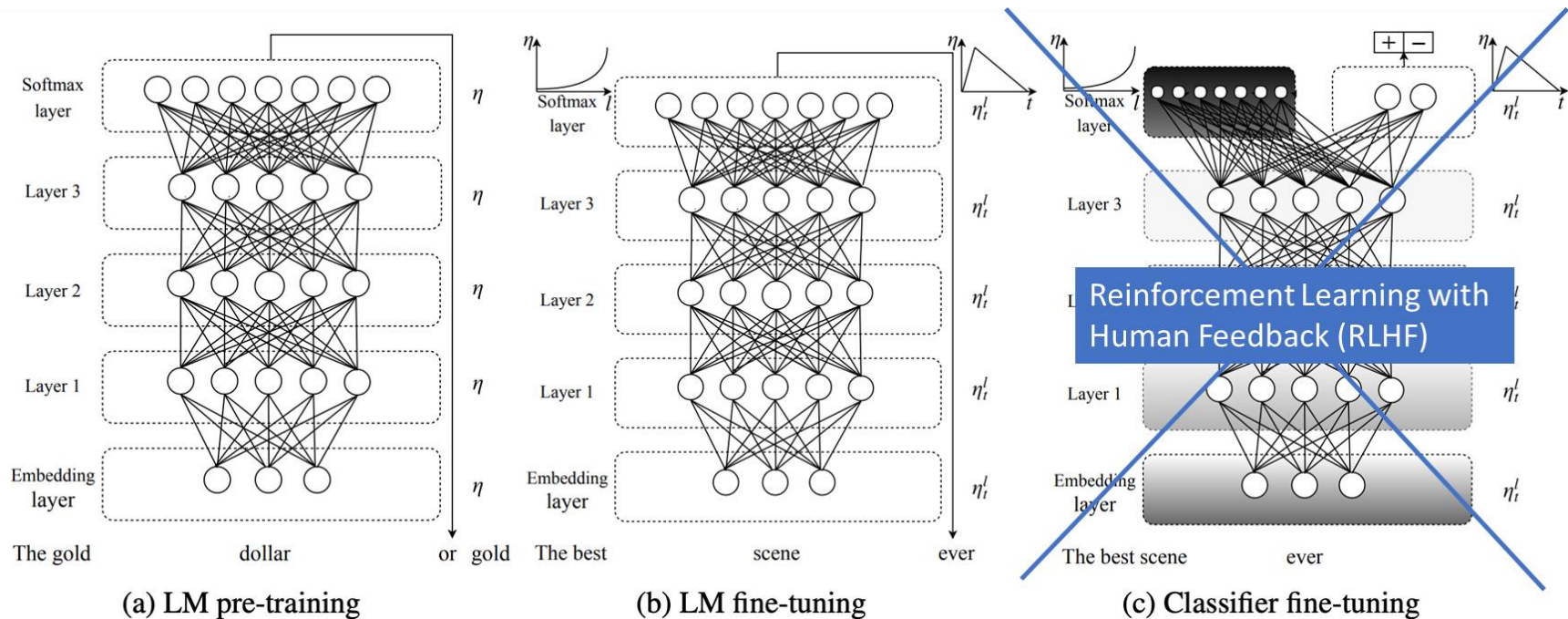
Language models like GPT-3 can write poetry, but they often amplify negative stereotypes. Researchers are trying different approaches to address the problem.

ChatGPT

ChatGPT is essentially a GPT3.5 or GPT4 **Large Language Model with constraints to prevent harmful messages** (e.g., hate speech) and inappropriate requests (e.g., how to build a bomb).



Base Model for ChatGPT

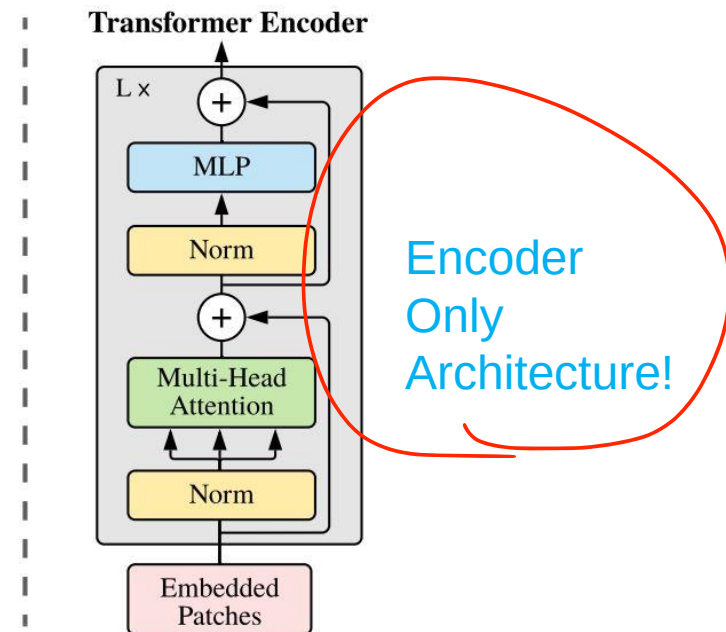
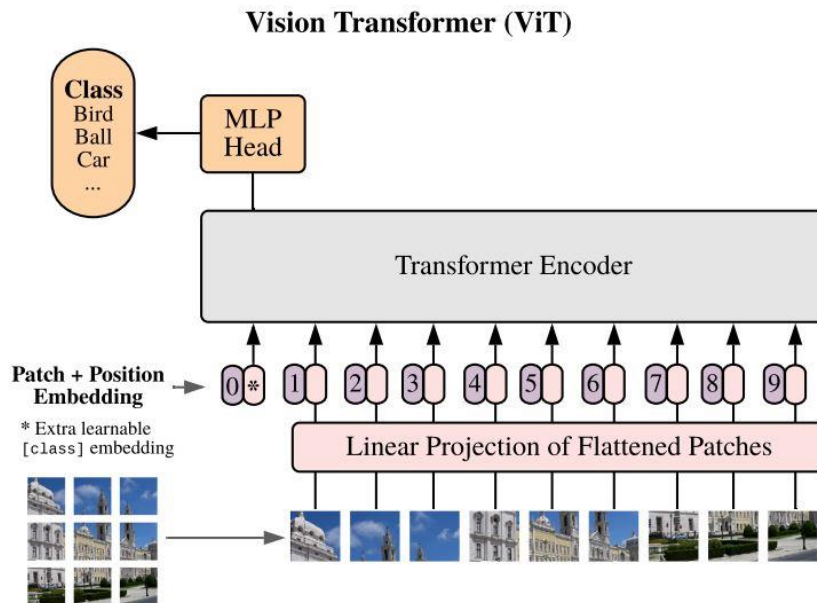


We will briefly introduce Reinforcement Learning next week

Transformers: NLP to Computer Vision

Vision Transformer (ViT)

- In 2020 a pure transformer model demonstrated better performance and greater efficiency than CNNs on image classification



Python and PyTorch Implementations



Transformers

<https://huggingface.co/transformers/>

Summary

- Adding **attention** allowed our models to “attend” to different parts of the input at every time step.
- **Transformers** are powerful architectures that use **self-attention** to achieve state-of-the-art performance.
 - It is highly **scalable** and highly parallelizable
 - **Faster** training, **larger** models, **better** performance across vision and language tasks
 - They are quickly replacing RNNs, LSTMs, and may even replace convolutions.
- Transformers are leading to a convergence of architectures in all domains.

Week 11 - Announcements

Assignment 4 – Sequential Data

- The final assignment
- Due ~~Tue, Nov 18th~~ **Fri, Nov 21 at 11pm**
- Part 1: Train an RNN to identify sentiment in movie reviews
- Part 2: Use a pretrained transformer model to identify sentiment in movie reviews
- Bonus: Explore generative LSTM to build a small language model to improve movie reviews. This is like the transfer learning seen before.

Presentations

- Due at 11pm on Sunday, Nov 30
- Presentations will take place:
Option 1: Monday (Dec 1) 9am – 11:30am (2.5 Hrs)
- Presentations will take place over Zoom
- Grading will be done by teaching team (75%) and your classmates (25%).

Next Time

Q/A Support Session

- Project

Lecture Week 12

- Interesting Topics (e.g., Reinforcement Learning)
- Ethics
- Fairness

Presentations

Final ExamProject Report / Guide